Contents

Key Concepts: 4
1.Supervised Learning: 4
1.1 Regression: 5
1.1.1 Linear regression: 5
Types of Linear Regression: 6
1.1.2 Polynomial Regression 7
1.1.3 Support Vector Regression (SVR) 8
1.1.4 Decision Tree 8
Common Splitting Criteria: 9
Ensemble Learning 10
Types of Ensemble Learning Method: 11
Advantages of Ensemble Learning: 12
Disadvantages: 12
1.1.5 Random forest 13
How Random Forest Works: 13
Key Parameters in Random Forest: 14
1.1.6 AdaBoost 14
How AdaBoost Works: 14
Advantages of AdaBoost: 14
Disadvantages of AdaBoost: 15
Hyperparameters in AdaBoost: 15
1.1.7 Gradient Boosting 15
How Gradient Boosting Works: 15
Key Parameters in Gradient Boosting: 16
1.1.8 eXtreme Gradient Boosting 17
How XGBoost Works: 17
Key Parameters in XGBoost: 17
Advantages of XGBoost: 17
Disadvantages of XGBoost: 18
1.1.9 CatBoost: 18
How CatBoost Works: 18

Key Parameters in CatBoost: 18
Advantages of CatBoost: 19
Disadvantages of CatBoost: 19
1.2 Classification: 20
Binary Classification 20
Multiclass Classification 20
Multilabel Classification 21
1.2.1 Logistic Regression 21
Key Concepts of Logistic Regression: 22
1.2.2 KNN 23
How KNN Works: 23
KNN for Classification 23
KNN for Regression 24
Advantages of KNN: 24
Disadvantages of KNN: 24
key parameters in KNN: 24
1.2.3 K-fold 24
How K-Fold Cross-Validation Works: 25
Advantages of K-Fold Cross-Validation: 25
Disadvantages of K-Fold Cross-Validation: 25
parameters in kfold: 25
1.2.5 Naive Bayes 27
Bayes' Theorem 27
How Naive Bayes Works 27
Advantages of Naive Bayes: 28
metrics for classification 28
unsupervised learning 28
Common Unsupervised Learning Techniques: 29
K-Means Clustering 29
Dimensionality Reduction: 30
PCA (Principal Component Analysis) 30
Outliers 31
Handling Outliers 31
Overfitting and Underfitting 32

Regularization 33
House price prediction problem 35
ENCODING 45
Scaling methods 50

MACHINE LEARNING

Machine Learning (ML) is a subset of artificial intelligence (AI) that focuses on developing algorithms that allow computers to learn from data and make decisions or predictions without being explicitly programmed. Essentially, it enables systems to improve their performance on a task over time based on experience.

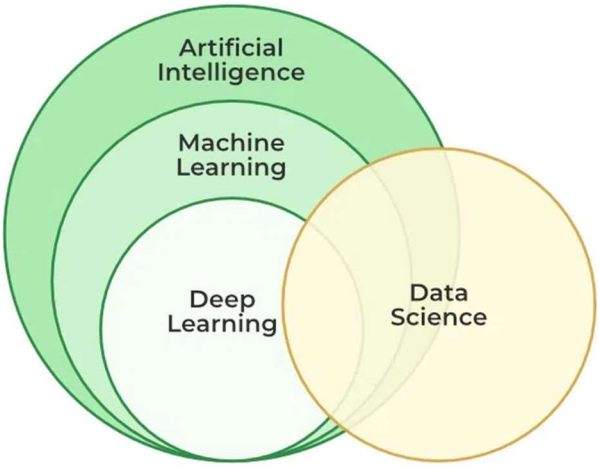
Key Concepts:

Supervised Learning

Unsupervised Learning

Reinforcement Learning

Semi-supervised Learning



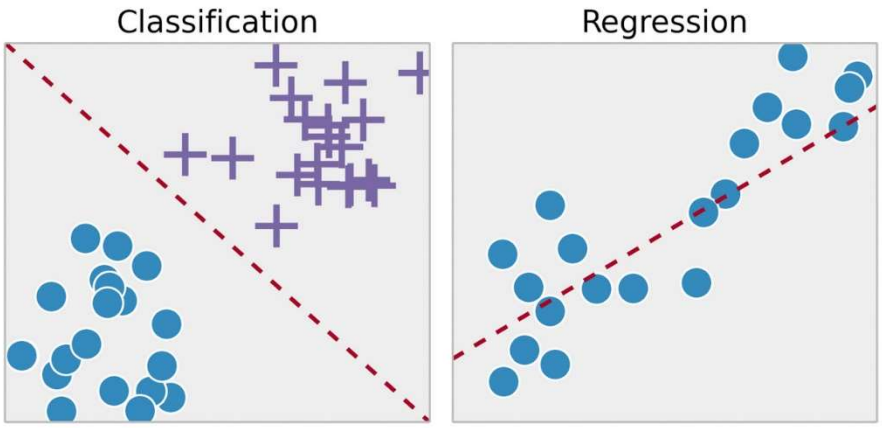
1.Supervised Learning:

Supervised Learning is a type of machine learning where the model is trained on labeled data. Each training example consists of an input and a corresponding output label. The goal is for the model to learn the mapping between the input and output, so it can predict the output for new, unseen inputs.

Types of Supervised Learning:

Regression

Classification



1.1 Regression:

Regression is a type of supervised learning in machine learning where the goal is to predict a continuous numeric value based on input features. It models the relationship between a dependent variable (what you want to predict) and one or more independent variables (the features).

Types of Regression:

Linear Regression

Polynomial Regression

Ridge and Lasso Regression

DecisionTree

RandomForest

AdaBoost

GraadientBoost

XGBoost

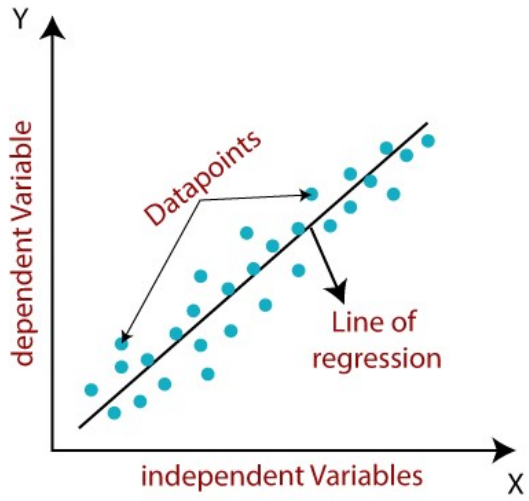
Cat Boost

Support Vector Regression (SVR)

1.1.1 Linear regression:

Linear regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable (target) and one or more independent variables (features). The objective of linear regression is to find the best-fitting line (or hyperplane)

that minimizes the difference between the actual values and the predicted values.



Types of Linear Regression:

Simple Linear Regression:

Models the relationship between two variables—one independent variable and one dependent variable. For simple linear regression, the equation of the model is:
$y=\beta 0+\beta 1x+\epsilon$

y: Dependent variable (target).

x: Independent variable (input feature)

β0: Intercept (the value of y when x=0).

$\beta 1$: Coefficient or slope of the line (how much y changes for a unit change in x).

ϵ: Error term (the difference between actual and predicted values).

Multiple Linear Regression:

Models the relationship between a dependent variable and two or more independent variables. For multiple linear regression, the model equation is:
$y=\beta 0+\beta 1x1+\beta 2x2+\cdots+\beta nxn+\epsilon$

Fitting the Model (Ordinary Least Squares - OLS):

the method used to estimate the coefficients ($\beta 0, \beta 1,\ldots,\beta n$) in linear regression. OLS works by minimizing the sum of squared differences between the actual and predicted values, i.e., minimizing the MSE.

parameters used in linear regression

Coefficients (Weights): A coefficient indicates how much the dependent variable changes for a one-unit change in the corresponding independent variable, assuming all other variables are held constant.

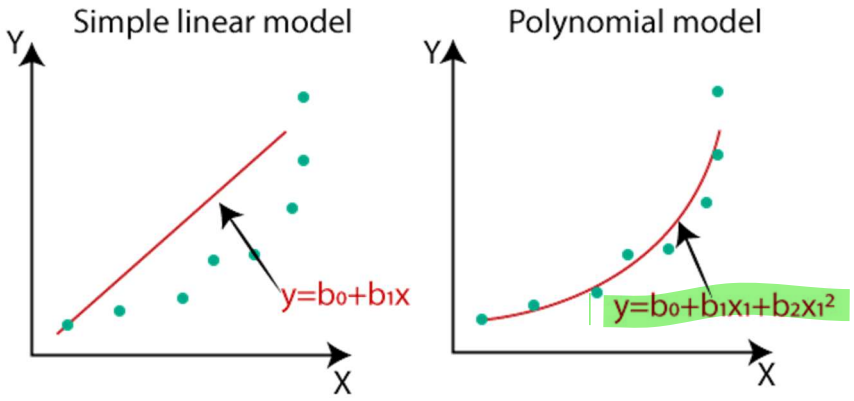
Intercept: The intercept is the value of the dependent variable when all independent variables (features) are zero. It's a constant that shifts the line (or hyperplane) vertically.

Learning Rate: The learning rate (α) is used when training the model using gradient descent (an optimization algorithm). It controls the size of the steps taken to update the coefficients during training. A smaller learning rate leads to smaller updates, allowing more precise convergence but at the cost of slower training. A larger learning rate speeds up training but risks overshooting the optimal solution.

Cost Function: The cost function measures how well the model's predictions match the actual values. In linear regression, the most commonly used cost function is Mean Squared Error (MSE), which quantifies the squared difference between predicted and actual values.

1.1.2 Polynomial Regression

Polynomial Regression is a type of regression analysis where the relationship between the dependent variable (target) and the independent variable(s) (features) is modeled as an n-th degree polynomial. Unlike linear regression, where the relationship between the variables is linear, polynomial regression can capture non-linear relationships by fitting a curve rather than a straight line. In polynomial regression, the model fits a polynomial function of degree n to the data. The general form of a polynomial regression model is :
$y=\beta 0+\beta 1x+\beta 2x2+\beta 3x3+\cdots+\beta nxn+\epsilon$



y is the dependent variable (target). x is the independent variable (input feature).$\beta 0$,$\beta 1$, … ,βn are the coefficients (parameters) to be estimated. ϵ is the error term (the difference between the predicted and actual values). n is the degree of the polynomial.

Polynomial regression is generally fit using ordinary least squares (OLS), the same as in linear regression. The difference is that the input features are expanded to include the polynomial terms, but the fitting process remains linear with respect to these new features.

Advantages of Polynomial Regression:

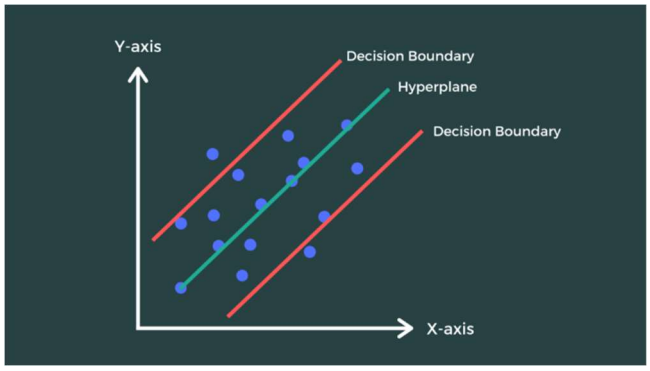
Polynomial regression allows you to model non-linear relationships between the independent and dependent variables. By increasing the degree of the polynomial, the model becomes more flexible and can fit complex patterns in the data.

Disadvantages of Polynomial Regression:

A high-degree polynomial can overfit the training data, capturing noise and small fluctuations rather than the true underlying relationship. Polynomial regression models can produce extreme values outside the range of the training data, making predictions unreliable for unseen data. Polynomial terms (especially of high degrees) can become highly correlated, which can lead to multicollinearity issues and instability in the coefficient estimates.

1.1.3 Support Vector Regression (SVR)

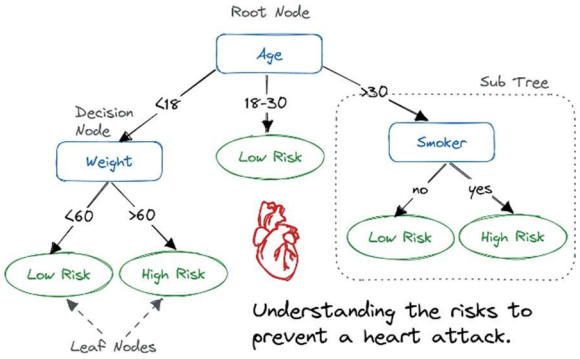
Support Vector Regression (SVR) is a type of regression algorithm that is based on the principles of Support Vector Machines (SVM). Unlike traditional linear regression, SVR attempts to fit the best line (or hyperplane) within a certain margin of tolerance to the data points. The key idea behind SVR is that it doesn't aim to minimize the error between the predicted and actual values directly, but rather it seeks to fit the best possible function that lies within a margin of tolerance (ϵ) while balancing the complexity of the model.



1.1.4 Decision Tree

A Decision Tree in Machine Learning (ML) is a popular supervised learning algorithm used for both classification and regression tasks. It models decisions and their possible consequences in a tree-like structure, where each internal node represents a feature (or

attribute), each branch represents a decision (outcome of a feature)



Splitting:

dividing a dataset into subsets at each node based on feature values to create child nodes. The goal of splitting is to find the feature and threshold that best separates the data to improve decision-making. At each node, the algorithm chooses a feature and a value (threshold) that divides the data into the most homogeneous subsets (minimizing impurity). The split decision is determined by an impurity measure such as Gini Impurity, Entropy (Information Gain), or Variance Reduction (for regression).

Common Splitting Criteria:

Gini Impurity (used in classification):

Measures how often a randomly chosen element would be incorrectly classified if it was randomly labeled.

Entropy & Information Gain:

Entropy measures the level of uncertainty or disorder.

Information Gain:

The reduction in entropy due to the split. It tells us how much information a feature gives us about the class.

Variance Reduction:

The split is chosen to minimize the variance of the target variable in the child nodes

Pruning:

Pruning reduces the size of a tree by removing sections that provide little power in predicting target variables. This is done to prevent overfitting. There are two types of pruning, Pre-pruning (Early stopping)-Stopping the tree growth early. Post-pruning-Cutting down the tree after it is fully grown.

Parameters in decision tree:

criterion: Determines the function to measure the quality of a split.

max_depth: The maximum depth of the tree. This controls how many levels the tree can grow. A deeper tree can model more complex patterns but may overfit the data. Setting this parameter limits the depth and prevents overfitting.

min_samples_split: The minimum number of samples required to split an internal node. Larger values prevent the model from learning too fine details and help reduce overfitting. Default is usually 2 (at least two samples required for a split).

min_samples_leaf: The minimum number of samples required to be at a leaf node. Setting a higher value ensures that each leaf node has enough samples, reducing the risk of overfitting.

max_features: The maximum number of features considered for splitting at each node. Helps in building a more generalized model by limiting the number of features evaluated.

min_impurity_decrease: A node will be split if the split induces a decrease in impurity greater than or equal to this value. Useful to control tree growth by requiring a minimum reduction in impurity to split a node.

max_leaf_nodes: Limits the number of leaf nodes in the tree. If not set, the tree can grow until the leaves contain fewer samples than min_samples_split.

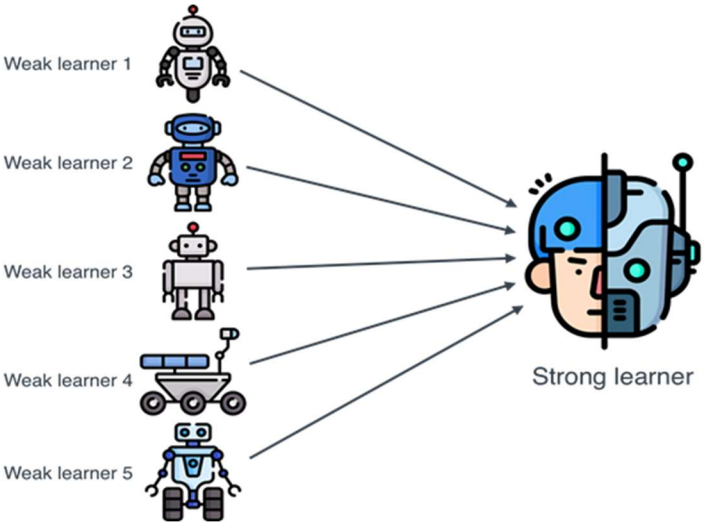
random_state: Controls the randomness of the estimator. This affects the randomness in splitting when splitter="random" and other randomized processes like bootstrapping in ensemble methods (e.g., Random Forest).

Ensemble Learning

Ensemble Learning is a machine learning technique that combines multiple models (often called "weak learners" or "base models") to create a more powerful model, leading to improved accuracy, robustness, and generalization. The idea is that by combining different models, we can reduce errors and achieve better predictive performance than any single model could achieve on its own.

Weak Learners: These are models that perform slightly better than random guessing (e.g., a decision tree with shallow depth). In ensemble learning, multiple weak learners are combined to create a strong learner.

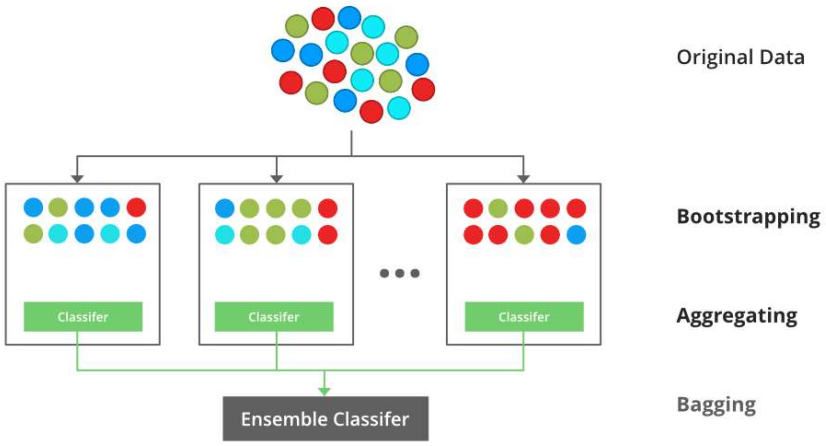
Strong Learner: The combined model, which is more accurate and robust than the individual weak learners.



Types of Ensemble Learning Method:

Bagging:

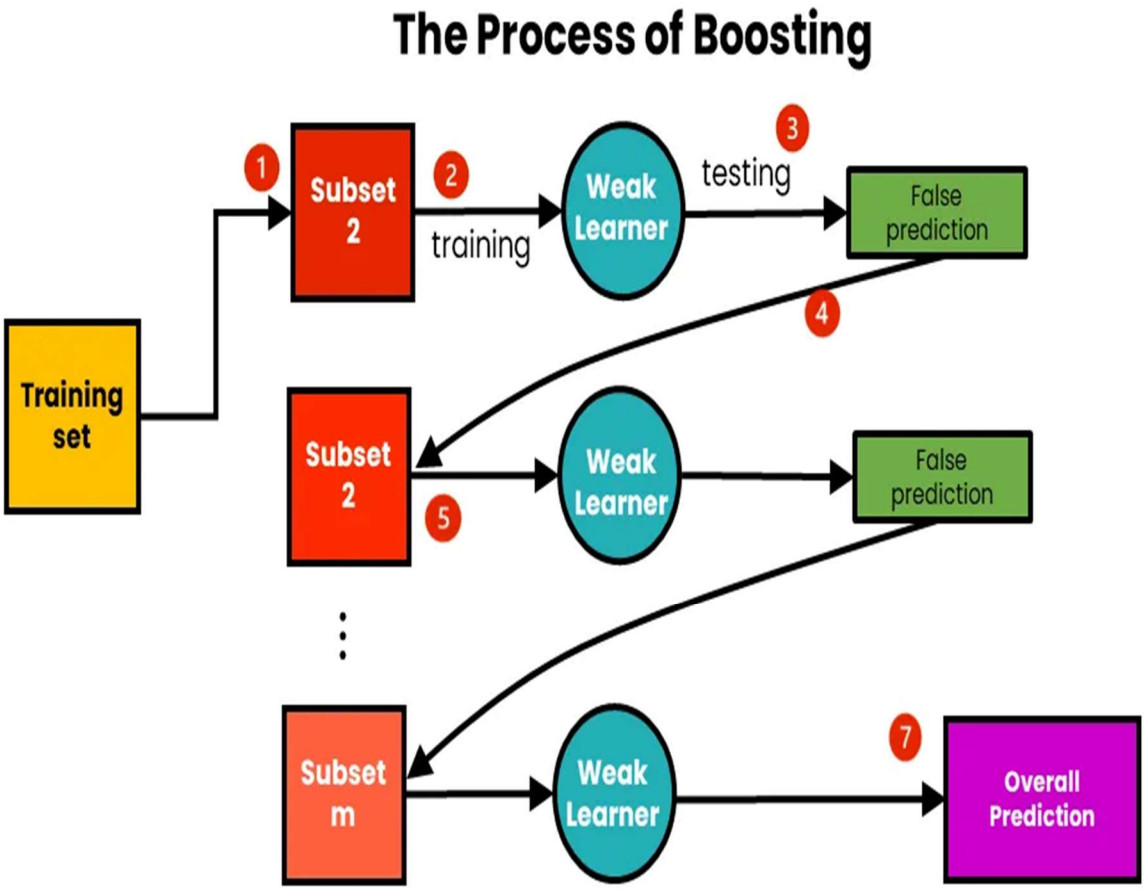
In Bagging, multiple models (usually decision trees) are trained on different random subsets of the training data (sampled with replacement). The final prediction is made by aggregating the predictions from all the models. some of the advantages of bagging are Reduces variance, helps prevent overfitting.



Example: Random Forest is a classic example of Bagging, where multiple decision trees are trained on bootstrapped samples.

Boosting:

Boosting builds models sequentially, where each model tries to correct the errors made by the previous models. It focuses on instances that are harder to classify, giving them more weight in subsequent models. boosting Reduces bias and variance, leading to better performance, especially on complex datasets.



Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.

Advantages of Ensemble Learning:

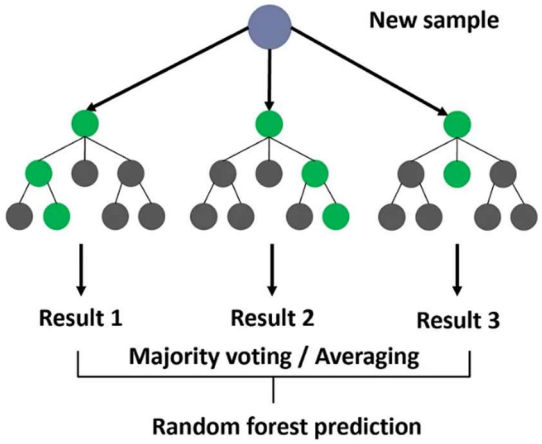
Especially in techniques like Bagging (e.g., Random Forest), overfitting is reduced as it smooths the predictions by averaging. By combining models, ensemble learning often results in higher accuracy and performance than individual models. Ensemble methods are more robust against errors and noise in the data because they aggregate multiple models' predictions. Works well with both classification and regression tasks.

Disadvantages:

Ensembles are more computationally expensive and harder to interpret than individual models. Training multiple models can be time-consuming, especially in Boosting and Stacking. Ensemble models may require more memory due to the need to store multiple models.

1.1.5 Random forest

Random Forest is an ensemble learning method that combines multiple decision trees to improve the model's accuracy, robustness, and generalization. It's used for both classification and regression tasks.



How Random Forest Works:

Multiple subsets of the original data are generated using bootstrapping (sampling with replacement). Each subset is used to train a separate decision tree. Each decision tree is constructed with a random subset of features at each split (instead of using all features), which prevents some features from dominating the prediction and creates uncorrelated trees. For Classification-The output of the Random Forest is determined by majority voting across the individual trees (i.e., the class most voted by the trees is chosen).For Regression-The final prediction is the average of the predictions from all the trees.

Advantages of Random Forest:

By averaging multiple trees, it smooths out the model, preventing overfitting that might occur with individual trees. It can work well with datasets with many features and provides a good estimate even with missing values. Random Forest provides a way to assess the importance of features based on how they improve the purity of splits in trees. Due to the averaging across multiple trees, Random Forest is resistant to overfitting on noisy data.

Disadvantages:

Training multiple trees can be computationally intensive, especially for large datasets. While individual decision trees are easy to interpret, Random Forest models (with many trees) lose interpretability. Since it averages across multiple trees, predictions can be slower compared to a single decision tree.

Key Parameters in Random Forest:

n_estimators: The number of decision trees in the forest. More trees typically lead to better performance but increase computational cost.

max_features: The number of features to consider when looking for the best split in each tree.

max_depth: The maximum depth of the trees. Shallower trees can prevent overfitting.

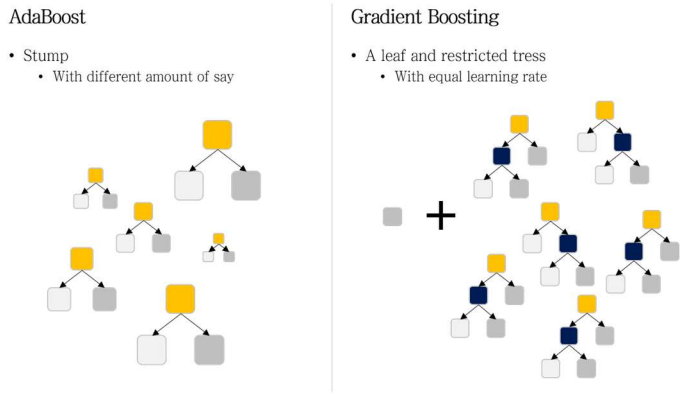
min_samples_split: The minimum number of samples required to split a node.

min_samples_leaf: The minimum number of samples required to be at a leaf node. Prevents overfitting in small datasets.

random_state: Sets the seed for reproducibility of the results.

1.1.6 AdaBoost

AdaBoost (Adaptive Boosting) is a popular ensemble learning technique used primarily for classification, though it can also be extended to regression. It works by combining multiple weak learners (typically simple models like decision stumps) to create a strong, more accurate model. The key idea behind AdaBoost is to sequentially train models, where each subsequent model focuses on correcting the mistakes made by the previous ones.



How AdaBoost Works:

Initialize weights for all training examples. Train a weak learner on the dataset. Calculate the error of the weak learner and assign it a weight based on accuracy. Update the weights of the training examples: Increase weights for misclassified samples and decrease them for correctly classified samples. Repeat for a set number of iterations. Combine the weak learners' predictions via a weighted majority vote.

Advantages of AdaBoost:

Easy to implement and typically requires little tuning of hyperparameters. AdaBoost focuses on the misclassified instances, helping the model learn from mistakes and

improving performance. In practice, AdaBoost is less prone to overfitting, even with many weak learners. Works well with various weak learners, though decision stumps are the most common. AdaBoost can deal with outliers and noisy data reasonably well, though Boosting methods like Gradient Boosting are generally more robust.

Disadvantages of AdaBoost:

While AdaBoost can handle noise to some extent, it may perform poorly if there are too many misclassified samples or noisy data points, as it focuses on them heavily. The sequential nature of the algorithm means that each weak learner depends on the previous one, which can lead to longer training times compared to methods like bagging (e.g., Random Forest). Overweighting difficult samples can sometimes degrade performance, especially with a very noisy dataset.

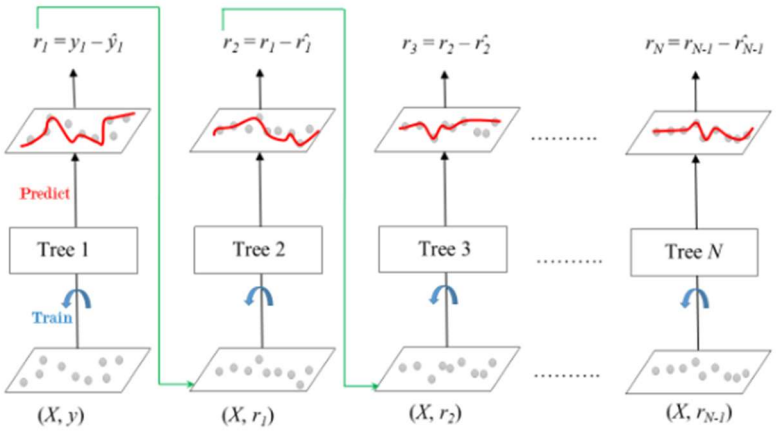
Hyperparameters in AdaBoost:

n_estimators: The number of weak learners to be trained (default is usually 50). A higher number can improve performance but also increases the risk of overfitting.

learning_rate: This scales the contribution of each weak learner. A smaller value can require more estimators but helps prevent overfitting.

1.1.7 Gradient Boosting

Gradient Boosting is a powerful and flexible ensemble learning technique used for both classification and regression tasks. It builds models sequentially, where each new model aims to correct the errors made by the previous models. Unlike AdaBoost, which assigns higher weights to misclassified samples, Gradient Boosting minimizes the overall prediction error using gradient descent.



How Gradient Boosting Works:

Start with an initial model F0(x), which could be as simple as predicting the mean (for regression) or the majority class (for classification). For each iteration, calculate the residual (error) between the true labels and the predictions of the current model. These

residuals represent the part of the data that the model is failing to predict accurately. Train a new weak learner (typically a decision tree) to predict the residuals or the negative gradient of the loss function. Add the predictions of the new weak learner to the current model. Repeat the process for a predefined number of iterations or until the error is sufficiently small. The final model is the sum of all weak learners, where each learner is weighted by the learning rate.

Key Parameters in Gradient Boosting:

n_estimators: The number of weak learners (trees) to be built. Too few learners may lead to underfitting, while too many can cause overfitting. This is typically tuned through cross-validation.

learning_rate: A regularization parameter that controls how much each weak learner contributes to the final model. Smaller values require more trees but can improve generalization.

max_depth: The maximum depth of the individual decision trees. Shallower trees can prevent overfitting, while deeper trees can capture more complex patterns but may overfit.

min_samples_split: The minimum number of samples required to split a node. Larger values make the model more conservative, preventing overfitting.

min_samples_leaf: The minimum number of samples in a leaf node. Helps in controlling overfitting.

subsample: The fraction of samples to be used for training each weak learner. Values less than 1.0 introduce randomness and help prevent overfitting (similar to bagging).

max_features: The number of features to consider when looking for the best split in trees. Reducing this can introduce randomness and improve generalization.

Advantages of Gradient Boosting:

Gradient Boosting often provides state-of-the-art performance in many applications, particularly for structured/tabular data. It reduces bias through sequential learning and reduces variance by aggregating multiple learners. You can optimize any differentiable loss function, making it very flexible. Gradient Boosting models can provide feature importance scores, helping with feature selection.

Disadvantages:

Gradient Boosting can be slow to train, especially for large datasets or deep trees. Without proper tuning (e.g., learning rate, number of estimators, regularization), it can overfit the training data. It has many hyperparameters that need to be carefully tuned to achieve optimal performance, requiring extensive cross-validation.

1.1.8 eXtreme Gradient Boosting

XGBoost (Extreme Gradient Boosting) is a highly efficient and scalable implementation of the gradient boosting framework. It is widely used in machine learning competitions and in industry due to its superior performance, speed, and flexibility compared to traditional gradient boosting techniques. XGBoost incorporates several optimizations and improvements that make it faster, more accurate, and capable of handling large datasets.

How XGBoost Works:

Initialize a model with a simple prediction (e.g., mean for regression, class probabilities for classification).Fit a weak learner (usually a decision tree) to the residual errors (the difference between actual and predicted values).Optimize using gradient descent to reduce the error, adjusting the weak learner based on the residuals. Add the weak learner's prediction to the overall model prediction. Repeat this process for a set number of iterations or until the error becomes sufficiently small. XGBoost introduces several improvements over standard gradient boosting by using Second-order derivative (Hessian) information to approximate the gradient during optimization. Regularized objective function to penalize overly complex trees. Shrinkage and subsampling to reduce overfitting.

Key Parameters in XGBoost:

n_estimators: The number of trees to build (similar to the number of iterations in boosting). More trees typically improve accuracy but may increase the risk of overfitting.

learning_rate: Controls how much each tree contributes to the final prediction. Smaller values (e.g., 0.01 or 0.1) make the model more robust to overfitting but require more trees.

max_depth: The maximum depth of each decision tree. Larger values allow the model to learn more complex patterns but may lead to overfitting. A value between 3 and 10 is common.

min_child_weight: The minimum sum of instance weights (hessian) needed in a child node. Larger values make the model more conservative, preventing it from learning overly specific patterns.

subsample: The proportion of the training data used to build each tree. Values less than 1.0 introduce randomness and help prevent overfitting.

Advantages of XGBoost:

XGBoost is optimized for both memory efficiency and processing time, making it ideal for large datasets. It supports both regression and classification tasks, as well as ranking, user-defined objectives, and custom loss functions. Built-in L1 and L2 regularization help prevent overfitting, making XGBoost robust to noisy datasets. XGBoost can handle missing data efficiently without the need for imputation. XGBoost provides feature importance scores, which can be used to understand the significance of each feature in the model.

Disadvantages of XGBoost:

XGBoost has many hyperparameters, and finding the optimal set can be time-consuming. While XGBoost is faster than traditional gradient boosting, it still requires significant training time compared to simpler models (especially for large datasets). If not properly regularized and tuned, XGBoost models can overfit, particularly with small datasets.

1.1.9 CatBoost:

CatBoost is a high-performance gradient boosting algorithm, designed specifically to handle categorical data effectively. It stands for Categorical Boosting and excels in situations where data contains a significant number of categorical features. Unlike traditional gradient boosting algorithms (like XGBoost or LightGBM), which require categorical features to be converted into numerical values (e.g., using one-hot encoding or label encoding), CatBoost inherently supports categorical variables without any manual preprocessing, making it highly efficient for datasets with mixed data types.

How CatBoost Works:

CatBoost follows the standard gradient boosting framework but introduces unique techniques to handle categorical data and improve performance. Categorical features are encoded automatically by the algorithm using target-based encoding, with techniques like mean encoding or frequency-based encoding to transform categories into numerical values in a robust manner. The boosting process in CatBoost follows a gradient boosting approach, but instead of making splits greedily based on the training data, CatBoost uses the ordered boosting technique to minimize overfitting. CatBoost builds symmetric trees where each level of the tree uses the same splitting rule, simplifying the model and speeding up both training and inference.Missing values are handled automatically by CatBoost, and you don't need to impute them manually before training. CatBoost effectively splits data by treating missing values as a separate category.

Key Parameters in CatBoost:

iterations: The number of trees (or boosting iterations) to build. A higher number usually increases model accuracy but may lead to overfitting.

learning_rate: Controls how much each tree contributes to the final prediction. A smaller learning rate requires more trees but often improves the model's performance.

depth: The maximum depth of the decision trees. Deeper trees can model more complex relationships but can also overfit. Typical values range from 4 to 10.

random_seed: Controls the randomness in data splitting, feature selection, and other aspects of the algorithm. Setting a random seed ensures reproducibility.

cat_features: A list of indices or feature names that represent categorical features. CatBoost automatically processes these features without needing manual encoding.

Advantages of CatBoost:

CatBoost shines in datasets with many categorical variables, automatically handling and encoding them effectively without manual intervention. CatBoost is more robust to overfitting than traditional boosting algorithms due to its ordered boosting technique and regularization strategies. CatBoost requires less manual tuning and often works well with default settings, making it accessible for users with varying levels of machine learning experience. With built-in support for GPU and CPU parallelization, CatBoost is faster than many other gradient boosting implementations. Missing values are managed efficiently, requiring no imputation steps before training the model. CatBoost can handle text features alongside numerical and categorical data, making it suitable for real-world datasets with mixed data types.

Disadvantages of CatBoost:

Although CatBoost is efficient, it may still be slower than LightGBM, particularly on large datasets where LightGBM's leaf-wise splitting provides a speed advantage. While CatBoost is easier to use out of the box, understanding its internals and advanced features (like ordered boosting) can be more complex compared to other boosting libraries. CatBoost can be memory-intensive when working with very large datasets, particularly when using GPU training.

Performance Metrics in Regression

Mean Absolute Error (MAE): MAE measures the average absolute difference between the predicted and actual values. It tells you how far the predictions are from the true values on average. Lower MAE means better performance. It's easy to interpret since it's in the same units as the output variable.

Mean Squared Error (MSE): MSE measures the average squared difference between the predicted and actual values. It penalizes larger errors more than MAE because errors are squared. Lower MSE indicates better model performance. Since errors are squared, MSE is more sensitive to outliers than MAE.

Root Mean Squared Error (RMSE): RMSE is the square root of the MSE. It provides an error value in the same unit as the output, making it easier to interpret than MSE. Lower RMSE means better performance. It's useful when large errors are more significant and need to be penalized.

R-squared: R^2 explains the proportion of variance in the dependent variable that is predictable from the independent variables. It measures how well the model fits the data. R^2 ranges from 0 to 1. A higher R^2 means a better fit. $R^2 = 1$ means perfect predictions. $R^2 = 0$ means the model explains none of the variance (the model is no better than using the mean value as a prediction).

Adjusted R-squared: Adjusted R^2 adjusts the R^2 value based on the number of predictors in the model. It penalizes the addition of irrelevant predictors that do not improve the

model's performance. Adjusted R^2 is useful when comparing models with different numbers of predictors. It increases only if the new predictor improves the model more than would be expected by chance.

1.2 Classification:

Classification is a type of supervised learning where the goal is to predict discrete class labels for given input data. It assigns inputs (features) to predefined categories or classes based on the model's learned patterns.

Types of Classification:

Binary Classification

Multiclass Classification

Multilabel Classification

common classification algorithms:

Logistic Regression

Decision Trees

Random Forest

Support Vector Machines (SVM)

K-Nearest Neighbors (KNN)

K-Fold

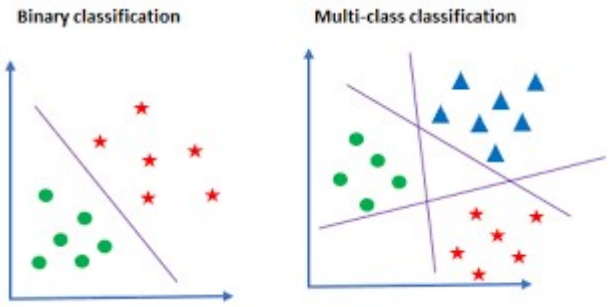
Naive Bayes

Binary Classification

binary classification is used when there are two possible outcomes to predict, and it is widely used in many real-world applications like spam filtering, medical diagnostics, and financial risk assessment.

Multiclass Classification

multiclass classification is used when the output belongs to one of multiple categories, and it is applied in areas like image recognition, language detection, and document classification. Models must handle the challenge of distinguishing among several classes, requiring specialized algorithms or techniques.



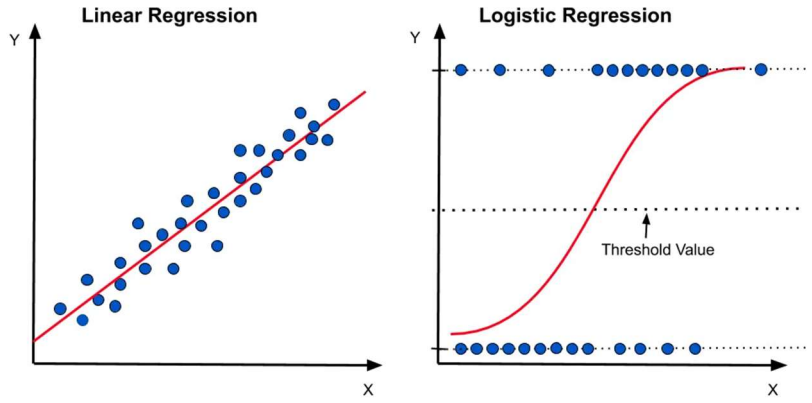
Strategies for Multiclass Classification: "One-vs-Rest (OvR)" The problem is broken into multiple binary classification tasks, where each class is treated as a "positive" class, and all other classes as "negative." "One-vs-One (OvO)" The algorithm builds a classifier for every pair of classes. For example, for 3 classes, 3 classifiers would be created to compare Class 1 vs Class 2, Class 1 vs Class 3, and Class 2 vs Class 3.

Multilabel Classification

multilabel classification allows for more flexibility in tasks where an instance might belong to multiple categories at the same time, and it is widely used in areas like text tagging, image annotation, and medical diagnosis.

1.2.1 Logistic Regression

Logistic Regression is a statistical method used for binary classification problems. Unlike linear regression, which is used to predict continuous outcomes, logistic regression predicts the probability of a binary outcome (i.e., 0 or 1, true or false, yes or no). It is a widely used algorithm in machine learning for classification tasks.



Key Concepts of Logistic Regression:

Binary Classification:

Logistic regression is primarily used for binary classification problems, where the target variable has two possible classes. For example Predicting if an email is spam (1) or not spam (0). Predicting if a patient has heart disease (1) or does not have heart disease (0).

Sigmoid Function:

Logistic regression uses a sigmoid (logistic) function to transform the output of a linear equation into a value between 0 and 1, representing the probability that a given instance belongs to a particular class. If the output is greater than 0.5, the instance is classified as belonging to class 1; otherwise, it is classified as belonging to class 0.

Cost Function (Log Loss):

instead of using the Mean Squared Error (MSE) as in linear regression, logistic regression uses the logistic loss function (also called log-loss or binary cross-entropy). The log-loss penalizes wrong predictions more harshly, making it suitable for classification tasks. This cost function is minimized using optimization techniques like gradient descent, which updates the model parameters (w and b) to find the best fit for the data.

Gradient Descent:

minimizes the cost function, logistic regression uses an optimization algorithm called gradient descent. The gradients of the cost function with respect to the model parameters (w and b) are computed, and the parameters are updated iteratively. The process is repeated until the cost function converges to a minimum, indicating that the model has learned the best set of parameters.

Multiclass Logistic Regression:

While logistic regression is typically used for binary classification, it can be extended to multiclass classification problems using methods like One-vs-Rest (OvR) or SoftMax Regression. In OvR, logistic regression is applied multiple times, once for each class, to classify data into more than two categories.

Regularization:

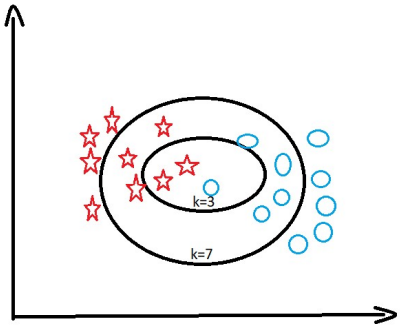
To prevent overfitting, logistic regression can be regularized by adding a penalty term to the cost function. The two common types of regularization are:

L2 Regularization (Ridge): Adds a penalty proportional to the sum of squared coefficients to the cost function. L1 Regularization (Lasso): Adds a penalty proportional to the sum of the absolute values of the coefficients.

Regularization helps control the complexity of the model by shrinking the coefficients, preventing them from becoming too large and thus overfitting the training data.

1.2.2 KNN

K-Nearest Neighbors (KNN) is one of the simplest and most intuitive supervised machine learning algorithms used for both classification and regression tasks. It works based on the idea of similarity or "closeness" between data points.



How KNN Works:

KNN doesn't have an explicit training phase like other algorithms (e.g., linear regression). It is considered a lazy learner, meaning it stores the entire dataset and does not learn a model from the data. Instead, KNN performs computation only when predictions are requested. When predicting the label for a new data point, KNN looks at the K nearest data points (neighbors) from the training set and bases its prediction on their labels. K is a hyperparameter that defines the number of nearest neighbors to consider for making predictions. Small K values- Low bias, but high variance (can be sensitive to noise).Large K values- High bias, but low variance (might generalize better but can lose local structure).

metrics to calculate "distance" between the data points:

Euclidean distance (most common): For numerical data.

Manhattan distance: For grid-like data structures.

Minkowski distance: A generalized version of the above two.

Hamming distance: For categorical data.

KNN for Classification

In classification tasks, KNN predicts the class of a new data point by looking at the K nearest neighbors and using a majority vote. For example, if K = 5 and among the 5 nearest neighbors, 3 belong to class A and 2 belong to class B, the new point will be classified as class A.

KNN for Regression

In regression tasks, KNN predicts the numeric value of a new data point by calculating the average value of the K nearest neighbors. For example, if K = 3 and the nearest neighbors have values [5, 6, 7], the predicted value will be the average, which is (5 + 6 + 7)/3 = 6.

Advantages of KNN:

KNN is easy to understand and implement. It doesn't make assumptions about the data (non-parametric) and simply stores the dataset. Can be used for both classification and regression. The decision process (based on neighbors) is intuitive and easy to explain.

Disadvantages of KNN:

Since it stores the entire dataset, prediction time increases with larger datasets. KNN requires storing all the data points, making it inefficient for large datasets. Outliers or noisy data can affect the model's performance, especially with small K values. As the number of features (dimensions) increases, the algorithm struggles to find relevant neighbors due to the sparsity of data in high-dimensional spaces.

key parameters in KNN:

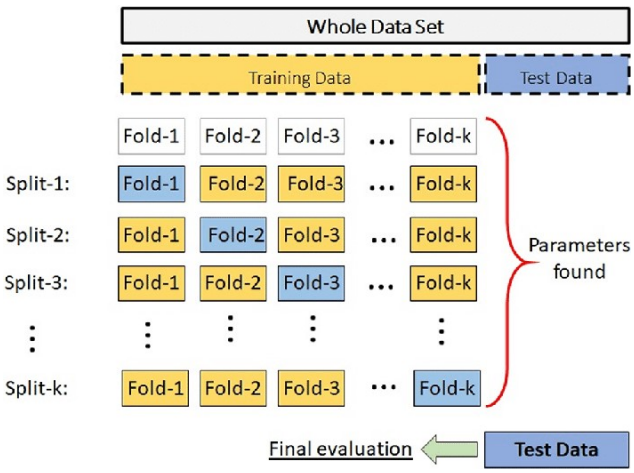
n_neighbors (k)- This parameter specifies the number of nearest neighbors to consider when making predictions.

weights-This parameter determines how much influence each neighbor has on the prediction. Using distance-based weighting can improve the model's performance when the distribution of data points is uneven.

leaf_size-This parameter is used to define the leaf size in the BallTree or KD-Tree algorithms. A smaller leaf size can lead to faster queries but may require more memory. The optimal value often depends on the dataset size and dimensionality.

1.2.3 K-fold

K-Fold Cross-Validation divides the dataset into K equally sized folds (subsets). The model is trained and evaluated K times, each time using a different fold as the validation set and the remaining K-1 folds as the training set. This process ensures that every observation in the dataset is used for both training and validation, which provides a robust estimate of the model's performance.



How K-Fold Cross-Validation Works:

The dataset is randomly shuffled and then divided into K equal-sized folds. For example, if K=5, the dataset is divided into 5 parts. For each fold Use the fold as the validation set. Use the remaining K-1 folds as the training set. Train the model on the training set and evaluate its performance on the validation set.

Advantages of K-Fold Cross-Validation:

It provides a more reliable estimate of model performance compared to a single train-test split, as every data point gets to be in a validation set exactly once. It helps in identifying models that generalize well and minimizes the likelihood of overfitting since the model is evaluated on multiple subsets. All samples are used for both training and validation, maximizing the available data for model training.

Disadvantages of K-Fold Cross-Validation:

Training and validating the model K times can be computationally expensive, especially for large datasets and complex models. The results can vary based on how the dataset is split into folds, so it's common to run K-Fold multiple times with different random seeds and average the results. Fold may not be appropriate for time series data, where the order of observations is important. In such cases, other techniques like Time Series Split should be used.

parameters in kfold:

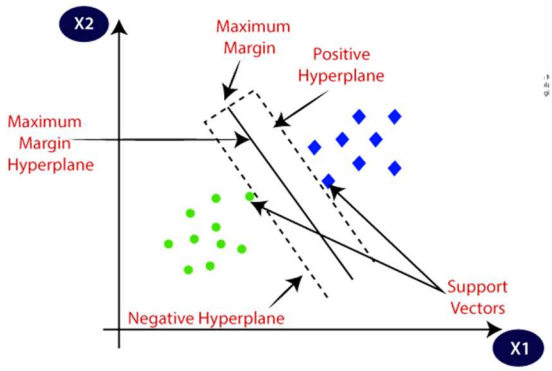
n_splits (K)- This is the most important hyperparameter in K-Fold Cross-Validation. It determines the number of folds (splits) in which the dataset is divided.

shuffle- This hyperparameter specifies whether to shuffle the data before splitting it into folds. By default, K-Fold does not shuffle the data. Shuffling is especially important for datasets with inherent ordering or when performing stratified K-Fold cross-validation.

Random_state - Controls the random number generator for shuffling the data when shuffle=True. Important when you want to ensure that the cross-validation process is reproducible, especially in experiments.

1.2.4 Support Vector Machines (SVM)

Support Vector Machine (SVM) is a powerful supervised machine learning algorithm used for both classification and regression tasks, although it is primarily used for classification. SVM is particularly effective in high-dimensional spaces and for cases where the number of features exceeds the number of samples. It works by finding the optimal hyperplane that best separates data points of different classes.



How SVM Works:

Find the optimal hyperplane that maximizes the margin between the classes. The critical data points that lie closest to the hyperplane and define the margin. Used to handle non-linear data by transforming it into a higher-dimensional space. Controls the trade-off between maximizing the margin and minimizing classification error. Once the model is trained, it predicts the class of new data points based on their position relative to the hyperplane.

Advantages of SVM :

Effective in high-dimensional spaces and when the number of features exceeds the number of samples. Memory efficient as it uses a subset of training points (support vectors). Works well for both linear and non-linear problems using kernels.

Disadvantages of SVM:

May be slow for very large datasets. Choosing the right kernel and its parameters (such as the regularization parameter CCC and kernel parameters) can be challenging. Not suitable for datasets with a lot of noise or overlapping classes.

Key Parameters to Tune in SVM:

C (Regularization parameter): Balances the trade-off between achieving a low error on training data and maintaining a large margin.

Gamma: Controls the influence of individual training examples in RBF and other kernels.

Kernel: Choose from linear, polynomial, RBF, or sigmoid, depending on your data.

1.2.5 Naive Bayes

Naive Bayes is a family of probabilistic algorithms based on applying Bayes' Theorem with a strong assumption: that the features are independent of each other given the class label. Despite the simplicity of this assumption (which is often not true in real-world data), Naive Bayes models perform well for various classification tasks, particularly in text classification, spam filtering, and sentiment analysis.

Bayes' Theorem

Bayes' Theorem provides a way to calculate the posterior probability of a hypothesis based on prior knowledge and evidence. P(A|B) = (P(B|A) ·P(A))/P(B) P(A|B): The posterior probability of event A (class) given event B (features).

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

P(B|A): The likelihood — the probability of event B occurring given that A is true.

P(A): The prior probability of event A (the initial belief before seeing the data).

P(B): The evidence or the total probability of event B occurring.

A is the class label (e.g., spam or not spam).

B is the set of features (e.g., words in an email).

How Naive Bayes Works

Calculate the prior probability for each class P(C). Calculate the likelihood P(xi|C) for each feature given the class. In Gaussian Naive Bayes, the likelihood is calculated using the mean and variance of the features for each class (assuming a Gaussian distribution).For Multinomial and Bernoulli Naive Bayes, the likelihood is calculated based on frequency counts or presence/absence of features.

For a new instance, calculate the posterior probability P(C|X) for each class based on the feature values X= {X1, X2,...,Xn}.The class with the highest posterior probability is chosen as the predicted class.

Advantages of Naive Bayes:

Naive Bayes is computationally efficient, with a linear time complexity in the number of features and data points.

metrics for classification

In classification tasks, performance metrics help evaluate how well the model classifies data into categories. These metrics are based on comparing the model's predictions with the actual labels. Here are the most commonly used performance metrics for classification

Accuracy: Accuracy measures the proportion of correctly classified instances (both positive and negative) out of the total number of instances. Higher accuracy indicates better performance, but it can be misleading in imbalanced datasets where one class dominates.

Precision: Precision, also known as positive predictive value, measures how many of the predicted positive instances are positive. High precision means fewer false positives. It is important when the cost of false positives is high (e.g., in spam detection).

Recall: Recall measures how many actual positive instances were correctly predicted by the model. High recall means fewer false negatives. It is important when missing positive instances is costly (e.g., in medical diagnosis).

F1 Score: The F1 score is the harmonic mean of precision and recall. It provides a balanced measure when there's a tradeoff between precision and recall. F1 score is useful when you need to balance precision and recall, especially in cases of imbalanced datasets.

Confusion Matrix: The confusion matrix is a table used to evaluate the performance of a classification model. It shows the actual versus predicted classifications for each class. The confusion matrix helps visualize errors such as false positives and false negatives, making it useful for understanding where the model is going wrong.

Area Under the ROC Curve (AUC-ROC): The AUC-ROC score summarizes the ROC curve into a single number. It represents the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative one. Range 0 to 1, where 1 indicates perfect classification and 0.5 indicates random guessing. Higher AUC means the model is better at distinguishing between positive and negative classes. It's useful for comparing models across different thresholds.

unsupervised learning

Unsupervised learning is a type of machine learning where the algorithm is provided with data that has no labeled outcomes or target variables. The goal is for the model to learn

patterns, structures, or relationships in the data on its own, without explicit instruction or supervision.

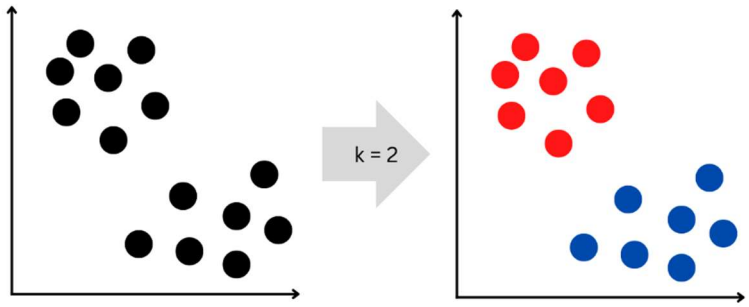
Unlike supervised learning, the input data in unsupervised learning is unlabeled, meaning that no outcomes or correct answers are provided. The model's objective is to discover hidden patterns or groupings in the data. This could include identifying clusters, associations, or dimensionality reduction. The model doesn't know if its predictions or categorizations are correct—it simply identifies inherent structures in the data.

Common Unsupervised Learning Techniques:

Clustering:

K-Means Clustering

K-Means Clustering is a popular unsupervised machine learning algorithm used for partitioning data into distinct groups (clusters). The aim is to group similar data points together in such a way that data points in the same cluster are more similar to each other than to those in other clusters. Each cluster is represented by its centroid, which is the average of all the points within that cluster.



K-Means Clustering Algorithm

How K-Means Works:

Start by specifying how many clusters you want, i.e., K. Randomly initialize K centroids (or use an optimized approach like K-Means++ to pick centroids far apart). Each data point is assigned to the nearest centroid based on a distance metric (usually Euclidean distance). This step forms K clusters. For each cluster, recalculate the centroid as the mean of all the points assigned to that cluster. The process of reassigning points to clusters and recomputing centroids is repeated until the centroids no longer change (or change minimally) or a predefined number of iterations is reached. The algorithm stops when the centroids stabilize or when a maximum number of iterations has been completed.

Hierarchical Clustering

Hierarchical Clustering is another popular unsupervised machine learning algorithm used to group similar objects into clusters. Unlike K-Means, which requires the number of clusters to be pre-defined, hierarchical clustering builds a hierarchy of clusters either from the bottom up (agglomerative) or from the top down (divisive), allowing for more flexibility.

How Hierarchical Clustering Work:

Initially, each data point is treated as its own cluster. Using a distance metric (like Euclidean distance), find the two clusters that are closest to each other. Merge the two closest clusters to form a single new cluster continue finding and merging the closest clusters until all data points are merged into a single large cluster. As the clusters are merged, a dendrogram is constructed to show the sequence of merges. Once the dendrogram is created, you can "cut" it at a certain level (based on height) to determine the number of clusters. The cut height corresponds to the distance between clusters.

A dendrogram is a tree-like diagram that illustrates the hierarchical relationships among the clusters. The y-axis represents the distance (dissimilarity) between clusters. The lower the merge in the dendrogram, the more similar the clusters are.

To decide on the number of clusters from hierarchical clustering, you can inspect the dendrogram and choose a "cut" point where the vertical distance between merged clusters is the largest (known as the largest gap). Large gaps in the dendrogram indicate a significant difference between clusters, so the number of clusters can be set by cutting at this gap. The clusters formed below this cut will be the final clusters.

Dimensionality Reduction:

PCA (Principal Component Analysis)

Principal Component Analysis (PCA) is a powerful unsupervised learning technique used for dimensionality reduction and data analysis. It helps to simplify datasets by transforming them into a lower-dimensional form while retaining most of the important information. PCA is commonly used to reduce the number of features (variables) in a dataset, making it easier to visualize and analyze, especially when the data has many correlated features.

Autoencoders

Autoencoders are a type of artificial neural network used for unsupervised learning, primarily for dimensionality reduction and data reconstruction. They work by learning to compress input data into a lower-dimensional representation (encoding) and then reconstruct it back to its original form (decoding). The main goal of an autoencoder is to learn a compact, informative representation of the input, making it an effective tool for tasks like noise reduction, anomaly detection, and feature extraction.

Autoencoders consist of two main parts, the encoder and the decoder, which are symmetric. The network has three main layers, Input Layer-The original input data (features). Hidden Layer(s) (Encoder)-The compressed or reduced representation of the data .Output Layer (Decoder)-The reconstructed output, which tries to match the original input.

The encoder maps the input into a smaller-dimensional latent space, and the decoder maps this compressed representation back into the original input space.

Advantages of Autoencoders:

Unlike PCA, autoencoders can learn complex non-linear mappings, making them more flexible. You can adjust the network's size, depth, and structure to suit specific datasets and tasks.

Limitations of Autoencoders:

Though autoencoders are unsupervised, their performance is usually evaluated using supervised methods Training an autoencoder requires a significant amount of data to avoid overfitting. If the autoencoder's capacity is too high (i.e., if the latent space is too large), the network might learn to copy the input instead of learning meaningful representations.

Challenges in Unsupervised Learning:

Since there are no labels, it's hard to quantitatively measure the performance of the model. Understanding the patterns found by unsupervised algorithms can be difficult. Some unsupervised algorithms, especially clustering and dimensionality reduction techniques, can be computationally intensive for large datasets.

Outliers

In machine learning, handling outliers is crucial for building robust models. Outliers can skew models, leading to inaccurate predictions or increased errors.

Handling Outliers

Simply remove outliers from the dataset. This is appropriate when you are confident that the outliers are noise or errors. Replace outliers with the nearest non-outlier value (for example, using percentiles like the 5th and 95th).Use transformations like log, square root, or box-cox transformations to reduce the impact of extreme values. Some models are robust to outliers, like decision trees and random forests. These models are less sensitive to extreme values compared to linear regression or K-means.

Here are some common methods used to detect and handle outliers:

Z-Score (Standard Score):

Measures how many standard deviations a data point is from the mean. If the Z-score is greater than a certain threshold (e.g., 3), the point is considered an outlier.

IQR (Interquartile Range) Method:

Outliers are detected based on the distribution of the data's quartiles. A common rule is to label data points outside the range:

[Q1−1.5×IQR, Q3+1.5×IQR]

Where IQR = Q3 - Q1, and Q1 and Q3 are the 1st and 3rd quartiles.

K-Nearest Neighbors (KNN) Distance: The average distance between a point and its k-nearest neighbors can be used to identify outliers. Larger distances correspond to outliers.

K-Means:

After clustering, points that are far from the cluster centroids or have a high distance to the closest cluster can be flagged as outliers.

Box Plot:

Box plots visualize the distribution of the data and show potential outliers as points that fall outside the whiskers (1.5 times the IQR).

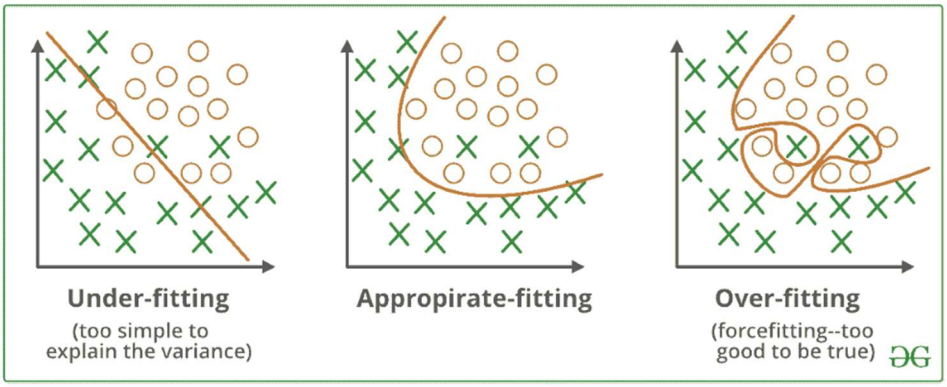
Scatter Plot:

Plotting data on a 2D or 3D scatter plot can help identify outliers visually, especially when combined with clustering or regression lines.

Overfitting and Underfitting

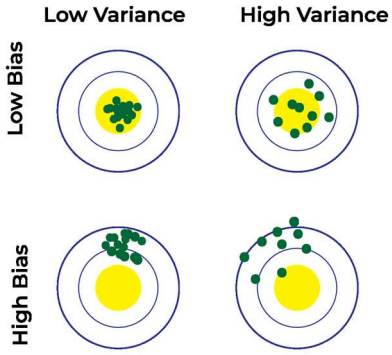
Overfitting is a phenomenon that occurs when a Machine Learning model is constrained to the training set and not able to perform well on unseen data. That is when our model learns the noise in the training data as well. This is the case when our model memorizes the training data instead of learning the patterns in it.

Underfitting on the other hand is the case when our model is not able to learn even the basic patterns available in the dataset. In the case of the underfitting model is unable to perform well even on the training data hence we cannot expect it to perform well on the validation data. This is the case when we are supposed to increase the complexity of the model or add more features to the feature set.



Bias refers to the errors which occur when we try to fit a statistical model on real-world data which does not fit perfectly well on some mathematical model. If we use a way too simplistic a model to fit the data then we are more probably face the situation of High Bias which refers to the case when the model is unable to learn the patterns in the data at hand and hence performs poorly.

Variance implies the error value that occurs when we try to make predictions by using data that is not previously seen by the model. There is a situation known as high variance that occurs when the model learns noise that is present in the data



Regularization

Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function, discouraging the model from assigning too much importance to individual features or coefficients.

regularization is used to prevent overfitting and balancing bias and variance and in handing multicollinearity.

One way to prevent overfitting is to use **regularization**, which penalizes large coefficients and constrains their magnitudes, thereby preventing a model from becoming overly complex and memorizing the training data instead of learning its underlying patterns.

Balancing Bias and Variance- Regularization can help balance the trade-off between model bias (underfitting) and model variance (overfitting) in machine learning, which leads to improved performance.

Handling Multicollinearity- When features are highly correlated (multicollinearity), regularization can stabilize the model by reducing coefficient sensitivity to small data changes.

The commonly used regularization techniques are :

Lasso Regularization – L1 Regularization

Ridge Regularization – L2 Regularization Elastic Net Regularization – L1 and L2 Regularization

Lasso Regression

A regression model which uses the L1 Regularization technique is called LASSO(Least Absolute Shrinkage and Selection Operator) regression. Lasso Regression adds the "absolute value of magnitude" of the coefficient as a penalty term to the loss function(L). Lasso regression also helps us achieve feature selection by penalizing the weights to approximately equal to zero if that feature does not serve any purpose in the model.

Ridge Regression

A regression model that uses the L2 regularization technique is called Ridge regression. Ridge regression adds the "squared magnitude" of the coefficient as a penalty term to the loss function(L)

Elastic Net Regression

This model is a combination of L1 as well as L2 regularization. That implies that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization

Benefits of Regularization

Regularization improves model performance by preventing overfitting, simplifying models, and making them more robust to noise and irrelevant features. By adding a penalty for large weights, regularization ensures that models generalize better to unseen data and work effectively, even in high-dimensional or noisy environments.

House price prediction problem

https://www.kaggle.com/c/house-prices-advanced-regression-techniques

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Importing the pandas library and assigns it the alias pd. Pandas is a powerful library for data manipulation and analysis. It provides data structures like DataFrames (for handling tabular data) and Series (for handling 1-dimensional data).Using pd, you can access all the functions and methods available in pandas, such as loading data from CSV files, manipulating datasets, performing analysis, etc.

Importing Numpy library and assigns it the alias np. NumPy is a fundamental library for scientific computing in Python. It provides support for working with arrays, matrices, and large datasets with fast mathematical operations. With np, you can perform numerical operations like generating arrays, reshaping data, performing element-wise operations, and matrix calculations.

Importing the Matplotlib library's pyplot module, which is commonly used for creating static, interactive, and animated plots in Python.matplotlib.pyplot provides functions for creating a wide variety of plots and charts such as line plots, bar charts, histograms, scatter plots, and more. Using plt, you can create visualizations to analyze and understand your data better.

When working in Jupyter notebooks, if you don't use %matplotlib inline, you would have to manually call plt.show() to display the plot. With this command, plots are automatically rendered inside the notebook, making the visualization workflow smoother. This command is specific to Jupyter environments like Jupyter Notebook and JupyterLab and is not needed in standard Python scripts.

```
housetrain=pd.read_csv("C:/Users/ankom/Downloads/train.csv")
housetest=pd.read_csv("C:/Users/ankom/Downloads/test.csv")
```

The pd.read_csv() function is used to load CSV files into pandas. It parses the CSV file into a DataFrame, which is a table-like data structure (rows and columns) that makes it easy to manipulate and analyze the data.

```
housetrain.shape
```

```
(1460, 81)
```

```
housetrain.columns
```

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
       'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
       'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
       'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt',
'YearRemodAdd',
       'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
       'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
       'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
       'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
       'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
       'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath',
'FullBath',
       'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
       'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu',
'GarageType',
       'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea',
'GarageQual',
       'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
       'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
       'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
       'SaleCondition', 'SalePrice'],
      dtype='object')
```

The attribute .shape is used to retrieve the dimensions (shape) of the dataframe object, which is most likely a pandas DataFrame (or potentially a NumPy array, since both share the .shape attribute). This attribute provides information about the number of rows and columns in the dataset.

```
housetrain.head()
```

```
   Id  MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape  \
   0   1          60       RL         65.0     8450   Pave   NaN      Reg
   1   2          20       RL         80.0     9600   Pave   NaN      Reg
   2   3          60       RL         68.0    11250   Pave   NaN      IR1
   3   4          70       RL         60.0     9550   Pave   NaN      IR1
   4   5          60       RL         84.0    14260   Pave   NaN      IR1

     LandContour Utilities  ... PoolArea PoolQC Fence MiscFeature MiscVal
   MoSold  \
   0         Lvl    AllPub  ...        0    NaN   NaN         NaN       0
   2
```

```
1          Lvl    AllPub  ...        0    NaN   NaN         NaN       0
5
2          Lvl    AllPub  ...        0    NaN   NaN         NaN       0
9
3          Lvl    AllPub  ...        0    NaN   NaN         NaN       0
2
4          Lvl    AllPub  ...        0    NaN   NaN         NaN       0
12

  YrSold  SaleType  SaleCondition  SalePrice
0   2008        WD         Normal     208500
1   2007        WD         Normal     181500
2   2008        WD         Normal     223500
3   2006        WD        Abnorml     140000
4   2008        WD         Normal     250000

[5 rows x 81 columns]
```

The .head() function in pandas is used to display the first few rows of a DataFrame or Series. By default, it shows the first 5 rows, but you can specify the number of rows you'd like to see.

```
house_train_num=housetrain.select_dtypes(include=[np.number])
house_train_cat=housetrain.select_dtypes(include=['object'])
```

The select_dtypes function is used to filter the DataFrame based on the type of data in each column. Here, include=[np.number] specifies that only columns with numerical data types should be selected. The object data type in pandas is commonly used for text (string) data or other non-numeric data types. By specifying include=['object'], you're instructing pandas to select all columns that contain data stored as objects, which typically represent categorical data.

```
house_train_num.describe().transpose()
```

```
                count          mean           std      min        25%  \
Id             1460.0     730.500000    421.610009      1.0     365.75
MSSubClass     1460.0      56.897260     42.300571     20.0      20.00
LotFrontage    1201.0      70.049958     24.284752     21.0      59.00
LotArea        1460.0   10516.828082   9981.264932   1300.0    7553.50
OverallQual    1460.0       6.099315      1.382997      1.0       5.00
OverallCond    1460.0       5.575342      1.112799      1.0       5.00
YearBuilt      1460.0    1971.267808     30.202904   1872.0    1954.00
YearRemodAdd   1460.0    1984.865753     20.645407   1950.0
```

1967.00					
MasVnrArea	1452.0	103.685262	181.066207	0.0	
0.00					
BsmtFinSF1	1460.0	443.639726	456.098091	0.0	
0.00					
BsmtFinSF2	1460.0	46.549315	161.319273	0.0	
0.00					
BsmtUnfSF	1460.0	567.240411	441.866955	0.0	
223.00					
TotalBsmtSF	1460.0	1057.429452	438.705324	0.0	
795.75					
1stFlrSF	1460.0	1162.626712	386.587738	334.0	
882.00					
2ndFlrSF	1460.0	346.992466	436.528436	0.0	
0.00					
LowQualFinSF	1460.0	5.844521	48.623081	0.0	
0.00					
GrLivArea	1460.0	1515.463699	525.480383	334.0	
1129.50					
BsmtFullBath	1460.0	0.425342	0.518911	0.0	
0.00					
BsmtHalfBath	1460.0	0.057534	0.238753	0.0	
0.00					
FullBath	1460.0	1.565068	0.550916	0.0	
1.00					
HalfBath	1460.0	0.382877	0.502885	0.0	
0.00					
BedroomAbvGr	1460.0	2.866438	0.815778	0.0	
2.00					
KitchenAbvGr	1460.0	1.046575	0.220338	0.0	
1.00					
TotRmsAbvGrd	1460.0	6.517808	1.625393	2.0	
5.00					
Fireplaces	1460.0	0.613014	0.644666	0.0	
0.00					
GarageYrBlt	1379.0	1978.506164	24.689725	1900.0	
1961.00					
GarageCars	1460.0	1.767123	0.747315	0.0	
1.00					
GarageArea	1460.0	472.980137	213.804841	0.0	
334.50					
WoodDeckSF	1460.0	94.244521	125.338794	0.0	
0.00					
OpenPorchSF	1460.0	46.660274	66.256028	0.0	
0.00					
EnclosedPorch	1460.0	21.954110	61.119149	0.0	
0.00					
3SsnPorch	1460.0	3.409589	29.317331	0.0	
0.00					
ScreenPorch	1460.0	15.060959	55.757415	0.0	

0.00					
PoolArea	1460.0	2.758904	40.177307	0.0	0.00
MiscVal	1460.0	43.489041	496.123024	0.0	0.00
MoSold	1460.0	6.321918	2.703626	1.0	5.00
YrSold	1460.0	2007.815753	1.328095	2006.0	2007.00
SalePrice	1460.0	180921.195890	79442.502883	34900.0	129975.00

	50%	75%	max
Id	730.5	1095.25	1460.0
MSSubClass	50.0	70.00	190.0
LotFrontage	69.0	80.00	313.0
LotArea	9478.5	11601.50	215245.0
OverallQual	6.0	7.00	10.0
OverallCond	5.0	6.00	9.0
YearBuilt	1973.0	2000.00	2010.0
YearRemodAdd	1994.0	2004.00	2010.0
MasVnrArea	0.0	166.00	1600.0
BsmtFinSF1	383.5	712.25	5644.0
BsmtFinSF2	0.0	0.00	1474.0
BsmtUnfSF	477.5	808.00	2336.0
TotalBsmtSF	991.5	1298.25	6110.0
1stFlrSF	1087.0	1391.25	4692.0
2ndFlrSF	0.0	728.00	2065.0
LowQualFinSF	0.0	0.00	572.0
GrLivArea	1464.0	1776.75	5642.0
BsmtFullBath	0.0	1.00	3.0
BsmtHalfBath	0.0	0.00	2.0
FullBath	2.0	2.00	3.0
HalfBath	0.0	1.00	2.0
BedroomAbvGr	3.0	3.00	8.0
KitchenAbvGr	1.0	1.00	3.0
TotRmsAbvGrd	6.0	7.00	14.0
Fireplaces	1.0	1.00	3.0
GarageYrBlt	1980.0	2002.00	2010.0
GarageCars	2.0	2.00	4.0
GarageArea	480.0	576.00	1418.0
WoodDeckSF	0.0	168.00	857.0
OpenPorchSF	25.0	68.00	547.0
EnclosedPorch	0.0	0.00	552.0
3SsnPorch	0.0	0.00	508.0
ScreenPorch	0.0	0.00	480.0
PoolArea	0.0	0.00	738.0
MiscVal	0.0	0.00	15500.0
MoSold	6.0	8.00	12.0

```
YrSold          2008.0    2009.00    2010.0
SalePrice      163000.0  214000.00  755000.0
```

By default, describe() outputs a DataFrame where each row represents a summary statistic (e.g., count, mean, etc.), and each column corresponds to a feature (numerical column).".transpose()" flips this structure. After transposing, the column names (features) will become the index (row names), and the summary statistics (count, mean, etc.) will become the columns.

```
house_train_num_corr=house_train_num.corr()
```

The .corr() method in pandas computes the correlation coefficient between numerical features (columns) in a DataFrame. The result is a correlation matrix, which is a table showing the correlation coefficients between pairs of variables. Correlation measures how two variables move in relation to each other.A positive correlation means that as one variable increases, the other tends to increase.A negative correlation means that as one variable increases, the other tends to decrease.A correlation of 0 means there is no linear relationship between the two variables.

By default, corr() in pandas calculates Pearson's correlation coefficient, which measures the linear relationship between variables. Pearson's correlation values range from -1 to 1: 1 means perfect positive correlation (variables move together in the same direction). -1 means perfect negative correlation (variables move in opposite directions). 0 means no linear correlation. Other correlation methods (like Kendall or Spearman) can also be used, but the default is Pearson.

```
house_train_num_corr["SalePrice"]
```

```
Id              -0.021917
MSSubClass      -0.084284
LotFrontage      0.351799
LotArea          0.263843
OverallQual      0.790982
OverallCond     -0.077856
YearBuilt        0.522897
YearRemodAdd     0.507101
MasVnrArea       0.477493
BsmtFinSF1       0.386420
BsmtFinSF2      -0.011378
BsmtUnfSF        0.214479
TotalBsmtSF      0.613581
1stFlrSF         0.605852
2ndFlrSF         0.319334
LowQualFinSF    -0.025606
GrLivArea        0.708624
BsmtFullBath     0.227122
BsmtHalfBath    -0.016844
FullBath         0.560664
HalfBath         0.284108
BedroomAbvGr     0.168213
```

```
KitchenAbvGr    -0.135907
TotRmsAbvGrd     0.533723
Fireplaces       0.466929
GarageYrBlt      0.486362
GarageCars       0.640409
GarageArea       0.623431
WoodDeckSF       0.324413
OpenPorchSF      0.315856
EnclosedPorch   -0.128578
3SsnPorch        0.044584
ScreenPorch      0.111447
PoolArea         0.092404
MiscVal         -0.021190
MoSold           0.046432
YrSold          -0.028923
SalePrice        1.000000
Name: SalePrice, dtype: float64
```

```
house_train_num_cols = []
house_train_num_cols.extend(house_train_num_corr[(house_train_num_corr["SaleP
rice"]>0.3) ].index.values)
house_train_num_cols.extend(house_train_num_corr[(house_train_num_corr["SaleP
rice"]<-0.3) ].index.values)
```

The code is filtering the features based on their correlation with SalePrice, both positively and negatively. It collects those features that are strongly correlated with SalePrice (either positively with correlation > 0.3, or negatively with correlation < -0.3). This list of features (house_train_num_cols) can later be used for further analysis, feature selection, or model training since they are the features most related to the target variable SalePrice.

```
house_train_num_cols
```

```
['LotFrontage',
 'OverallQual',
 'YearBuilt',
 'YearRemodAdd',
 'MasVnrArea',
 'BsmtFinSF1',
 'TotalBsmtSF',
 '1stFlrSF',
 '2ndFlrSF',
 'GrLivArea',
 'FullBath',
 'TotRmsAbvGrd',
 'Fireplaces',
 'GarageYrBlt',
 'GarageCars',
 'GarageArea',
 'WoodDeckSF',
```

```
 'OpenPorchSF',
 'SalePrice']

h_train_num_col_filtered=house_train_num[house_train_num_cols]

h_train_num_col_filtered.head()
```

	LotFrontage	OverallQual	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1
0	65.0	7	2003	2003	196.0	706
1	80.0	6	1976	1976	0.0	978
2	68.0	7	2001	2002	162.0	486
3	60.0	7	1915	1970	0.0	216
4	84.0	8	2000	2000	350.0	655

	TotalBsmtSF	1stFlrSF	2ndFlrSF	GrLivArea	FullBath	TotRmsAbvGrd
0	856	856	854	1710	2	8
1	1262	1262	0	1262	2	6
2	920	920	866	1786	2	6
3	756	961	756	1717	1	7
4	1145	1145	1053	2198	2	9

	Fireplaces	GarageYrBlt	GarageCars	GarageArea	WoodDeckSF	OpenPorchSF
0	0	2003.0	2	548	0	61
1	1	1976.0	2	460	298	0
2	1	2001.0	2	608	0	42
3	1	1998.0	3	642	0	35
4	1	2000.0	3	836	192	84

	SalePrice
0	208500
1	181500
2	223500
3	140000
4	250000

```
house_train_num.isnull().sum().sort_values(ascending=False)
```

```
LotFrontage      259
GarageYrBlt       81
```

```
MasVnrArea         8
Id                 0
OpenPorchSF        0
KitchenAbvGr       0
TotRmsAbvGrd       0
Fireplaces         0
GarageCars         0
GarageArea         0
WoodDeckSF         0
EnclosedPorch      0
HalfBath           0
3SsnPorch          0
ScreenPorch        0
PoolArea           0
MiscVal            0
MoSold             0
YrSold             0
BedroomAbvGr       0
FullBath           0
MSSubClass         0
BsmtFinSF2         0
LotArea            0
OverallQual        0
OverallCond        0
YearBuilt          0
YearRemodAdd       0
BsmtFinSF1         0
BsmtUnfSF          0
BsmtHalfBath       0
TotalBsmtSF        0
1stFlrSF           0
2ndFlrSF           0
LowQualFinSF       0
GrLivArea          0
BsmtFullBath       0
SalePrice          0
dtype: int64
```

".isnull()" This checks for missing values (NaN) in the DataFrame. It returns a DataFrame of the same shape, where each element is True if the corresponding value is missing, and False if it is not.".sum()" This sums up the True values (which are interpreted as 1) along the columns (by default, sum() works along the columns in a DataFrame). As a result, for each column in Dataframe, it calculates how many missing values are present. ".sort_values(ascending=False)" Finally, this sorts the resulting series of missing values counts in descending order (ascending=False). This means that columns with the most missing values will appear first, followed by those with fewer or no missing values.

```
for hc in ["LotFrontage","GarageYrBlt","MasVnrArea"]:
    print (hc)
```

```
    print(house_train_num[hc].mean())
    print(house_train_num[hc].median())
```

```
LotFrontage
70.04995836802665
69.0
GarageYrBlt
1978.5061638868744
1980.0
MasVnrArea
103.68526170798899
0.0
```

df_name[column_name].mean(): This calculates and prints the mean (average) value of the column from the DataFrame.

df_name[column_name].median(): This calculates and prints the median value of the column. The median is the middle value of the dataset when the values are ordered from lowest to highest.

```
for col in ["LotFrontage","GarageYrBlt","MasVnrArea"]:

h_train_num_col_filtered[col].fillna(h_train_num_col_filtered[col].median(),i
nplace=True)
```

```
C:\Users\ankom\AppData\Local\Temp\ipykernel_20932\1583005769.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

h_train_num_col_filtered[col].fillna(h_train_num_col_filtered[col].median(),i
nplace=True)
```

fillna(): This function is used to fill missing values (NaN) in the specified column. In this case, the function is replacing the NaN values with the median of that particular column.inplace=True modifies the column in place, meaning the changes are made directly to the DataFrame, without needing to assign it to a new variable.

```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
house_train_cat.head()
```

	MSZoning	Street	Alley	LotShape	LandContour	Utilities	LotConfig	LandSlope
0	RL	Pave	NaN	Reg	Lvl	AllPub	Inside	Gtl
1	RL	Pave	NaN	Reg	Lvl	AllPub	FR2	Gtl
2	RL	Pave	NaN	IR1	Lvl	AllPub	Inside	Gtl
3	RL	Pave	NaN	IR1	Lvl	AllPub	Corner	Gtl
4	RL	Pave	NaN	IR1	Lvl	AllPub	FR2	Gtl

```
  Neighborhood Condition1  ... GarageType GarageFinish GarageQual
GarageCond  \
0      CollgCr       Norm  ...     Attchd          RFn         TA
TA
1      Veenker      Feedr  ...     Attchd          RFn         TA
TA
2      CollgCr       Norm  ...     Attchd          RFn         TA
TA
3      Crawfor       Norm  ...     Detchd          Unf         TA
TA
4      NoRidge       Norm  ...     Attchd          RFn         TA
TA

  PavedDrive PoolQC Fence MiscFeature SaleType SaleCondition
0          Y    NaN   NaN         NaN       WD        Normal
1          Y    NaN   NaN         NaN       WD        Normal
2          Y    NaN   NaN         NaN       WD        Normal
3          Y    NaN   NaN         NaN       WD       Abnorml
4          Y    NaN   NaN         NaN       WD        Normal

[5 rows x 43 columns]
```

from sklearn.preprocessing import LabelEncoder This imports the LabelEncoder class from the sklearn.preprocessing module. LabelEncoder is used to convert categorical labels (non-numeric, text-based categories) into numerical form. It is helpful when you have categorical data that needs to be converted into a machine-readable numeric format for machine learning model

ENCODING

In machine learning, handling categorical data is crucial, especially when working with models that require numerical input. Several encoding techniques are available to transform categorical features into a format that can be fed into machine learning models.

Label Encoding

Label Encoding is a technique used to convert categorical (non-numeric) data into numeric form by assigning a unique integer to each category. For example, if a column contains categories like "Red", "Blue", and "Green", Label Encoding will convert them to 0, 1, and 2, respectively. It is useful when the categorical variable has an ordinal relationship (e.g., "Low", "Medium", "High"). However, it can introduce problems with non-ordinal data, as models may mistakenly assume a meaningful order between the encoded values. It is implemented using LabelEncoder from sklearn.preprocessing.

One-Hot Encoding

One-Hot Encoding is a technique used to convert categorical data into a numerical format by creating binary (0 or 1) columns for each unique category. Each category becomes its

own column, and for each data point, a 1 is placed in the column corresponding to its category, while all other columns are set to 0.For example, if you have a feature "Color" with categories "Red", "Blue", and "Green", One-Hot Encoding would create three new columns: Color_Red, Color_Blue, and Color_Green. If a row has the category "Red", the Color_Red column gets a 1, and the other two get 0. Binary Encoding

Target Encoding (Mean Encoding)

Target Encoding (or Mean Encoding) replaces categorical values with the mean of the target variable for each category. For example, in a binary classification problem, each category is replaced by the average probability of the target being 1 for that category. This method captures the relationship between the categorical feature and the target variable, making it useful for supervised learning. However, it can lead to overfitting, especially when there are few data points for a category, so techniques like smoothing or cross-validation are often applied to mitigate this. Target encoding works well with high-cardinality features.

Ordinal Encoding

Ordinal Encoding is a technique used to convert categorical variables into numerical values based on their inherent order or rank. Each category is assigned an integer value that reflects its position in a meaningful sequence. For example, a feature with values like "Low", "Medium", and "High" could be encoded as 0, 1, and 2, respectively. This method is ideal for features where the categories have a clear, ordered relationship (e.g., education level, customer satisfaction). However, it assumes that the difference between the encoded numbers is meaningful, which may not always be the case.

```
house_train_cat.isnull().sum().sort_values(ascending=False)
```

```
PoolQC          1453
MiscFeature     1406
Alley           1369
Fence           1179
MasVnrType       872
FireplaceQu      690
GarageType        81
GarageCond        81
GarageQual        81
GarageFinish      81
BsmtFinType2      38
BsmtExposure      38
BsmtFinType1      37
BsmtQual          37
BsmtCond          37
Electrical         1
KitchenQual        0
CentralAir         0
Functional         0
HeatingQC          0
```

```
Heating             0
PavedDrive          0
SaleType            0
MSZoning            0
Street              0
Condition2          0
LotShape            0
LandContour         0
Utilities           0
LotConfig           0
LandSlope           0
Neighborhood        0
Condition1          0
BldgType            0
Foundation          0
HouseStyle          0
RoofStyle           0
RoofMatl            0
Exterior1st         0
Exterior2nd         0
ExterQual           0
ExterCond           0
SaleCondition       0
dtype: int64
```

```
for col in ["PoolQC","MiscFeature","Alley","Fence","FireplaceQu"]:
    house_train_cat[col].fillna('No Value',inplace=True)
```

```
for col in
["GarageCond","GarageQual","GarageFinish","GarageType","BsmtFinType2","BsmtEx
posure","BsmtFinType1","BsmtQual","BsmtCond","MasVnrType","Electrical"]:

house_train_cat[col].fillna(house_train_cat[col].value_counts().idxmax(),inpl
ace=True)
```

.idxmax() returns the index (category) with the highest frequency, i.e., the most frequent value or mode of the column.inplace=True modifies the column directly within the DataFrame, so the changes are applied to the original data without needing to reassign it to a new variable.

```
house_train_cat1=house_train_cat.apply(le.fit_transform)
```

The apply() function in pandas applies a given function to each column (or row) of the DataFrame.le.fit_transform applies the LabelEncoder instance (le), which was initialized earlier, to each column of the DataFrame.fit_transform() first fits the LabelEncoder to the unique categories in the column (i.e., learns the mapping from categories to numbers) and then transforms the categorical values into their respective numeric labels.

```
house_train_cat1.head()
```

```
   MSZoning  Street  Alley  LotShape  LandContour  Utilities  LotConfig  \
0         3       1      1         3            3          0          4
1         3       1      1         3            3          0          2
2         3       1      1         0            3          0          4
3         3       1      1         0            3          0          0
4         3       1      1         0            3          0          2

   LandSlope  Neighborhood  Condition1  ...  GarageType  GarageFinish  \
0          0             5           2  ...           1             1
1          0            24           1  ...           1             1
2          0             5           2  ...           1             1
3          0             6           2  ...           5             2
4          0            15           2  ...           1             1

   GarageQual  GarageCond  PavedDrive  PoolQC  Fence  MiscFeature
SaleType  \
0           4           4           2       3      4            1
8
1           4           4           2       3      4            1
8
2           4           4           2       3      4            1
8
3           4           4           2       3      4            1
8
4           4           4           2       3      4            1
8

   SaleCondition
0              4
1              4
2              4
3              0
4              4

[5 rows x 43 columns]
```

```
from sklearn.linear_model import LinearRegression
Linreg=LinearRegression()
housetraindf2=pd.concat([h_train_num_col_filtered,house_train_cat1],axis=1)

housetraindf2.head()
```

```
   LotFrontage  OverallQual  YearBuilt  YearRemodAdd  MasVnrArea  BsmtFinSF1
\
0         65.0            7       2003          2003       196.0
706
1         80.0            6       1976          1976         0.0
978
2         68.0            7       2001          2002       162.0
486
```

```
3         60.0            7       1915          1970         0.0
216
4         84.0            8       2000          2000       350.0
655

   TotalBsmtSF  1stFlrSF  2ndFlrSF  GrLivArea  ...  GarageType
GarageFinish  \
0          856       856       854       1710  ...           1
1
1         1262      1262         0       1262  ...           1
1
2          920       920       866       1786  ...           1
1
3          756       961       756       1717  ...           5
2
4         1145      1145      1053       2198  ...           1
1

   GarageQual  GarageCond  PavedDrive  PoolQC  Fence  MiscFeature
SaleType  \
0           4           4           2       3      4            1
8
1           4           4           2       3      4            1
8
2           4           4           2       3      4            1
8
3           4           4           2       3      4            1
8
4           4           4           2       3      4            1
8

   SaleCondition
0              4
1              4
2              4
3              0
4              4

[5 rows x 62 columns]
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
dff = scaler.fit_transform(housetraindf2)

dff
```

imports the StandardScaler class from the sklearn.preprocessing module. StandardScaler is used to standardize (scale) the features of the dataset by removing the mean and scaling to unit variance.creates an instance of the StandardScaler class, named scaler. This instance will be used to perform the scaling operation. The StandardScaler computes the mean and

standard deviation of each feature (column) in the dataset, which will be used to standardize the data.fit() Calculates the mean and standard deviation for each feature in housetraindf2.transform() Uses those calculated statistics to scale the data. Each feature in housetraindf2 is transformed by subtracting the mean and dividing by the standard deviation. As a result, each feature will have a mean of 0 and a standard deviation of 1, which is the essence of standardization.

Scaling methods are essential in machine learning, particularly when dealing with algorithms that are sensitive to the scale of input features. Different scaling methods can help improve the performance and convergence speed of models.

Scaling methods

Standardization (Z-score Normalization):

This method scales the data so that the distribution of the feature has a mean of 0 and a standard deviation of 1. Useful for algorithms that assume a Gaussian distribution, such as Logistic Regression, Linear Regression, and Support Vector Machines.

Min-Max Scaling (Normalization)

This method scales the data to a fixed range, typically [0, 1]. The minimum value becomes 0 and the maximum value becomes 1.Useful when you want to ensure all features contribute equally and when the data does not follow a Gaussian distribution. Commonly used in neural networks.

Robust Scaling

This method uses the median and the interquartile range (IQR) for scaling, making it robust to outliers. Particularly effective when dealing with datasets that contain many outliers.

MaxAbs Scaling

This method scales the data by dividing by the maximum absolute value of each feature, resulting in values between -1 and 1. Useful when the data is already centered at zero and you want to maintain the sparsity of the data (often used with sparse datasets).

Logarithmic Scaling

This method applies the logarithm function to the feature values, which can help reduce skewness in the data. Useful for features with exponential distributions or when handling large ranges of values.

```
housetraindf1 = pd.DataFrame(data=dff, columns=housetraindf2.columns)

housetraindf1.head()
```

```
   LotFrontage  OverallQual  YearBuilt  YearRemodAdd  MasVnrArea  BsmtFinSF1
   \
```

```
0    -0.220875     0.651479   1.050994      0.878668    0.514104
0.575425
1     0.460320    -0.071836   0.156734     -0.429577   -0.570750
1.171992
2    -0.084636     0.651479   0.984752      0.830215    0.325915
0.092907
3    -0.447940     0.651479  -1.863632     -0.720298   -0.570750   -
0.499274
4     0.641972     1.374795   0.951632      0.733308    1.366489
0.463568

   TotalBsmtSF  1stFlrSF  2ndFlrSF  GrLivArea  ...  GarageType
GarageFinish  \
0    -0.459303 -0.793434  1.161852   0.370333  ...   -0.686776     -
0.281887
1     0.466465  0.257140 -0.795163  -0.482512  ...   -0.686776     -
0.281887
2    -0.313369 -0.627826  1.189351   0.515013  ...   -0.686776     -
0.281887
3    -0.687324 -0.521734  0.937276   0.383659  ...    1.586905
0.950313
4     0.199680 -0.045611  1.617877   1.299326  ...   -0.686776     -
0.281887

   GarageQual  GarageCond  PavedDrive    PoolQC     Fence  MiscFeature  \
0    0.215646    0.186019    0.289745  0.063305  0.457447    -0.185975
1    0.215646    0.186019    0.289745  0.063305  0.457447    -0.185975
2    0.215646    0.186019    0.289745  0.063305  0.457447    -0.185975
3    0.215646    0.186019    0.289745  0.063305  0.457447    -0.185975
4    0.215646    0.186019    0.289745  0.063305  0.457447    -0.185975

   SaleType  SaleCondition
0  0.313867       0.208502
1  0.313867       0.208502
2  0.313867       0.208502
3  0.313867      -3.426284
4  0.313867       0.208502

[5 rows x 62 columns]
```

```
X1=housetraindf1.drop(["SalePrice"],axis=1)
y=housetraindf2["SalePrice"]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X1, y,
test_size=0.3,random_state=42)
house_test_num=housetest.select_dtypes(include=[np.number])
house_test_cat=housetest.select_dtypes(include=['object'])
house_train_num_cols.remove('SalePrice')
```

```
h_test_num_col_filtered=house_test_num[house_train_num_cols]
(h_test_num_col_filtered.isnull().sum().sort_values(ascending=False))
```

```
LotFrontage     227
GarageYrBlt      78
MasVnrArea       15
GarageArea        1
BsmtFinSF1        1
TotalBsmtSF       1
GarageCars        1
TotRmsAbvGrd      0
WoodDeckSF        0
Fireplaces        0
GrLivArea         0
FullBath          0
OverallQual       0
2ndFlrSF          0
1stFlrSF          0
YearRemodAdd      0
YearBuilt         0
OpenPorchSF       0
dtype: int64
```

"train_test_split" This is a function from scikit-learn that splits arrays or matrices into random train and test subsets. It's typically used to split your features (X1) and target labels (y) into separate training and testing datasets.X1- Represents the input features (predictor variables).y- Represents the target variable (what you're trying to predict).test_size=0.3- Indicates that 30% of the data will be used for testing, while 70% will be used for training.

X_train: The training subset of the input features.

X_test: The testing subset of the input features.

y_train: The training subset of the target variable.

y_test: The testing subset of the target variable.

```
for hc in
["LotFrontage","GarageYrBlt","MasVnrArea","TotalBsmtSF","GarageArea","GarageC
ars","BsmtFinSF1"]:
    print (hc)
    print(h_test_num_col_filtered[hc].mean())
    print(h_test_num_col_filtered[hc].median())
```

```
LotFrontage
68.58035714285714
67.0
GarageYrBlt
1977.7212165097756
1979.0
```

```
MasVnrArea
100.70914127423822
0.0
TotalBsmtSF
1046.1179698216736
988.0
GarageArea
472.76886145404666
480.0
GarageCars
1.7661179698216736
2.0
BsmtFinSF1
439.2037037037037
350.5
```

```
for col in
["LotFrontage","GarageYrBlt","MasVnrArea","TotalBsmtSF","GarageArea","GarageC
ars","BsmtFinSF1"]:

h_test_num_col_filtered[col].fillna(h_test_num_col_filtered[col].median(),inp
lace=True)
```

```
C:\Users\ankom\AppData\Local\Temp\ipykernel_20932\496194425.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

h_test_num_col_filtered[col].fillna(h_test_num_col_filtered[col].median(),inp
lace=True)
```

```
house_test_cat.isnull().sum().sort_values(ascending=False)
```

```
PoolQC          1456
MiscFeature     1408
Alley           1352
Fence           1169
MasVnrType       894
FireplaceQu      730
GarageCond        78
GarageQual        78
GarageFinish      78
GarageType        76
BsmtCond          45
BsmtQual          44
BsmtExposure      44
BsmtFinType2      42
BsmtFinType1      42
MSZoning           4
```

```
Utilities          2
Functional         2
Exterior2nd        1
KitchenQual        1
SaleType           1
Exterior1st        1
LotShape           0
LandContour        0
PavedDrive         0
LotConfig          0
LandSlope          0
Neighborhood       0
Condition1         0
Condition2         0
Electrical         0
CentralAir         0
HeatingQC          0
Heating            0
BldgType           0
HouseStyle         0
RoofStyle          0
RoofMatl           0
Street             0
Foundation         0
ExterCond          0
ExterQual          0
SaleCondition      0
dtype: int64
```

```
for col in ["PoolQC","MiscFeature","Alley","Fence","FireplaceQu"]:
    house_test_cat[col].fillna('No Value',inplace=True)
```

```
for col in
["GarageCond","GarageQual","GarageFinish","GarageType","BsmtQual","BsmtCond",
"BsmtFinType2","BsmtExposure","BsmtFinType1","MasVnrType","MSZoning","Utiliti
es","Functional","KitchenQual","SaleType","Exterior2nd","Exterior1st"]:

house_test_cat[col].fillna(house_test_cat[col].value_counts().idxmax(),inplac
e=True)

house_test_cat.isnull().sum().sort_values(ascending=False)
```

```
MSZoning         0
FireplaceQu      0
BsmtFinType1     0
BsmtFinType2     0
Heating          0
HeatingQC        0
CentralAir       0
Electrical       0
KitchenQual      0
```

```
Functional       0
GarageType       0
BsmtCond         0
GarageFinish     0
GarageQual       0
GarageCond       0
PavedDrive       0
PoolQC           0
Fence            0
MiscFeature      0
SaleType         0
BsmtExposure     0
BsmtQual         0
Street           0
Condition2       0
Alley            0
LotShape         0
LandContour      0
Utilities        0
LotConfig        0
LandSlope        0
Neighborhood     0
Condition1       0
BldgType         0
Foundation       0
HouseStyle       0
RoofStyle        0
RoofMatl         0
Exterior1st      0
Exterior2nd      0
MasVnrType       0
ExterQual        0
ExterCond        0
SaleCondition    0
dtype: int64
```

```
house_test_cat1=house_test_cat.apply(le.fit_transform)
```

```
housetestdf1=pd.concat([h_test_num_col_filtered,house_test_cat1],axis=1)
```

pd.concat(...) function is from the pandas library and is used to concatenate DataFrames or Series along a specified axis.axis=1 This parameter specifies that the concatenation should be performed along the columns (i.e., horizontally)

```
housetestdf1.head()
```

```
   LotFrontage  OverallQual  YearBuilt  YearRemodAdd  MasVnrArea  BsmtFinSF1
   \
0         80.0            5       1961          1961         0.0
   468.0
1         81.0            6       1958          1958       108.0
```

```
923.0
2         74.0            5       1997          1998         0.0
791.0
3         78.0            6       1998          1998        20.0
602.0
4         43.0            8       1992          1992         0.0
263.0

   TotalBsmtSF  1stFlrSF  2ndFlrSF  GrLivArea  ...  GarageType
GarageFinish  \
0        882.0       896         0        896  ...           1
2
1       1329.0      1329         0       1329  ...           1
2
2        928.0       928       701       1629  ...           1
0
3        926.0       926       678       1604  ...           1
0
4       1280.0      1280         0       1280  ...           1
1

   GarageQual  GarageCond  PavedDrive  PoolQC  Fence  MiscFeature
SaleType  \
0           3           4           2       2      2            1
8
1           3           4           2       2      4            0
8
2           3           4           2       2      2            1
8
3           3           4           2       2      4            1
8
4           3           4           2       2      4            1
8

   SaleCondition
0              4
1              4
2              4
3              4
4              4

[5 rows x 61 columns]
```

```
from sklearn.tree import DecisionTreeRegressor
DecTree=DecisionTreeRegressor()
from sklearn.ensemble import RandomForestRegressor
RandFor=RandomForestRegressor(n_estimators=5000)
from sklearn.ensemble import GradientBoostingRegressor
GBM=GradientBoostingRegressor(n_estimators=3000)
```

from sklearn.tree import DecisionTreeRegressor imports the DecisionTreeRegressor class from the sklearn.tree module.DecTree = DecisionTreeRegressor() creates an instance of the DecisionTreeRegressor.

from sklearn.ensemble import RandomForestRegressor imports the RandomForestRegressor class from the sklearn.ensemble module.RandFor = RandomForestRegressor(n_estimators=5000) creates an instance of the RandomForestRegressor with n_estimators set to 5000.

from sklearn.ensemble import GradientBoostingRegressor imports the GradientBoostingRegressor class from the sklearn.ensemble module.GBM = GradientBoostingRegressor(n_estimators=3000) creates an instance of the GradientBoostingRegressor with n_estimators set to 3000.

```
LM=Linreg.fit(X_train,y_train)
LM.score(X_train,y_train)
```

```
0.847483282992682
```

```
dt_M=DecTree.fit(X_train,y_train)
dt_M.score(X_train,y_train)
```

```
1.0
```

```
rf_M=RandFor.fit(X_train,y_train)
rf_M.score(X_train,y_train)
```

```
0.9799298184372554
```

```
gb_M=GBM.fit(X_train,y_train)
gb_M.score(X_train,y_train)
```

```
0.9999934457491977
```

```
from sklearn.metrics import accuracy_score, r2_score
```

```
gb_y_pred = gb_M.predict(X_test)
rf_y_pred = rf_M.predict(X_test)
lm_y_pred = LM.predict(X_test)
```

```
r2_score(y_test,gb_y_pred)
```

```
0.8397536315682296
```

```
r2_score(y_test,rf_y_pred)
```

```
0.8440957484674279
```

```
r2_score(y_test,lm_y_pred)
```

```
0.8062130822988727
```

```
y_eval=gb_M.predict(housetestdf1)
```

```
DecTree_0=DecisionTreeRegressor()
RandFor_0=RandomForestRegressor(n_estimators=5000)

pd.DataFrame(y_eval).to_csv('gbm.csv')

from sklearn.model_selection import KFold

test_scores={}
train_scores={}
for k in range (2,3,2):
    kf = KFold(n_splits=k)
    sum_train = 0
    sum_test = 0
    data = housetraindf1
    for train, test in kf.split(data):
        #train_data = np.array(data)[train]
        #test_data = np.array(data)[test]
        train_data = data.iloc[train,:]
        test_data = data.iloc[test,:]
        x_train = train_data.drop(["SalePrice"],axis=1)
        y_train = train_data["SalePrice"]
        x_test = test_data.drop(["SalePrice"],axis=1)
        y_test = test_data["SalePrice"]
        RandFor_0_model = RandFor_0.fit(x_train,y_train)
        sum_train += RandFor_0_model.score(x_train,y_train)
        y_pred = RandFor_0_model.predict(x_test)
        sum_test += r2_score(y_test,y_pred)
    average_test = sum_test/k
    average_train = sum_train/k
    test_scores[k] = average_test
    train_scores[k] = average_train

test_scores

{2: 0.8524936377987989}

train_scores

{2: 0.9773184915608579}

test_scores={}
train_scores={}
for k in range (2,7,2):
    kf = KFold(n_splits=k)
    sum_train = 0
    sum_test = 0
    data = housetraindf1
    for train, test in kf.split(data):
        #train_data = np.array(data)[train]
        #test_data = np.array(data)[test]
        train_data = data.iloc[train,:]
        test_data = data.iloc[test,:]
```

```
        x_train = train_data.drop(["SalePrice"],axis=1)
        y_train = train_data["SalePrice"]
        x_test = test_data.drop(["SalePrice"],axis=1)
        y_test = test_data["SalePrice"]
        RandFor_0_model = RandFor_0.fit(x_train,y_train)
        sum_train += RandFor_0_model.score(x_train,y_train)
        y_pred = RandFor_0_model.predict(x_test)
        sum_test += r2_score(y_test,y_pred)
    average_test = sum_test/k
    average_train = sum_train/k
    test_scores[k] = average_test
    train_scores[k] = average_train

test_scores
```

```
{2: 0.8524438716541959, 4: 0.8621819925155777, 6: 0.8619013897752694}
```

```
train_scores
```

```
{2: 0.976690684566738, 4: 0.9802061183824848, 6: 0.9809095397754284}
```

```
def train_test_evaluate_with_kFold(train_data,kmax,algo):
  test_scores={}
  train_scores={}
  for k in range (2,kmax,2):
    kf = KFold(n_splits=k, shuffle=True, random_state=42)
    sum_train = 0
    sum_test = 0
    #data = housetraindf1
    data = train_data
    for train, test in kf.split(data):
        #train_data = np.array(data)[train]
        #test_data = np.array(data)[test]
        train_data = data.iloc[train,:]
        test_data = data.iloc[test,:]
        x_train = train_data.drop(["SalePrice"],axis=1)
        y_train = train_data["SalePrice"]
        x_test = test_data.drop(["SalePrice"],axis=1)
        y_test = test_data["SalePrice"]
        algo_model = algo.fit(x_train,y_train)
        sum_train += algo_model.score(x_train,y_train)
        y_pred = algo_model.predict(x_test)
        sum_test += r2_score(y_test,y_pred)
    average_test = sum_test/k
    average_train = sum_train/k
    test_scores[k] = average_test
    train_scores[k] = average_train
    print("kvalue: ",k)
  return (train_scores,test_scores)

print(train_test_evaluate_with_kFold(housetraindf1,11,RandFor_0))
```

```
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
({2: 0.9772164258499022, 4: 0.9776983043842336, 6: 0.9804249040998271, 8:
0.9819772428541946, 10: 0.9810786905088879}, {2: 0.8521502464249937, 4:
0.8294724660579271, 6: 0.8585955496046164, 8: 0.868915750589635, 10:
0.8546832445303542})
```

```
DecTree_0=DecisionTreeRegressor()
GBM_0=GradientBoostingRegressor(n_estimators=3000)
LinReg_0=LinearRegression()
```

```
algo_dict =
{"LinReg":LinReg_0,"DecTree":DecTree_0,"GBM":GBM_0,"RF":RandFor_0}
algo_train_scores={}
algo_test_scores={}
```

```
max_kfold = 10
for algo_name in algo_dict.keys():
    print(algo_name)
    train_score, test_score =
train_test_evaluate_with_kFold(housetraindf1,max_kfold+1,algo_dict[algo_name]
)
    algo_train_scores[algo_name]=train_score
    algo_test_scores[algo_name]=test_score
```

```
print(algo_train_scores)
print(algo_test_scores)
```

```
LinReg
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
DecTree
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
GBM
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
RF
kvalue:  2
```

```
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
{'LinReg': {2: 0.8515525568159341, 4: 0.8798918160406497, 6:
0.8892507857704975, 8: 0.9194573118782721, 10: 0.9231709101403272},
'DecTree': {2: 1.0, 4: 1.0, 6: 1.0, 8: 1.0, 10: 1.0}, 'GBM': {2:
0.9999996532619156, 4: 0.9999999944087113, 6: 0.9999999995081273, 8:
0.9999999999655694, 10: 0.9999999999940308}, 'RF': {2: 0.9771660142510799, 4:
0.9776807204080697, 6: 0.9803871025489408, 8: 0.9819891863425078, 10:
0.9810284844832248}}
{'LinReg': {2: -8.940759370531631e+23, 4: -339.63965657330226, 6:
0.5283525768819836, 8: 0.8699774994416338, 10: 0.8608249354369647},
'DecTree': {2: 0.7248010420575637, 4: 0.7456029266855135, 6:
0.7317766062896904, 8: 0.6950929667968263, 10: 0.7304250128575125}, 'GBM':
{2: 0.852207408312825, 4: 0.8207779786766694, 6: 0.8598491807254968, 8:
0.8726513296710203, 10: 0.8756668310654433}, 'RF': {2: 0.8522112132825009, 4:
0.8300274157428527, 6: 0.8585850619937085, 8: 0.8682587199600716, 10:
0.8543036684661217}}
```

```
from matplotlib import style

algo_train_scores["GBM"].values()

dict_values([0.9999996532619156, 0.9999999944087113, 0.9999999995081273,
0.9999999999655694, 0.9999999999940308])

list(algo_train_scores["GBM"].values())
```

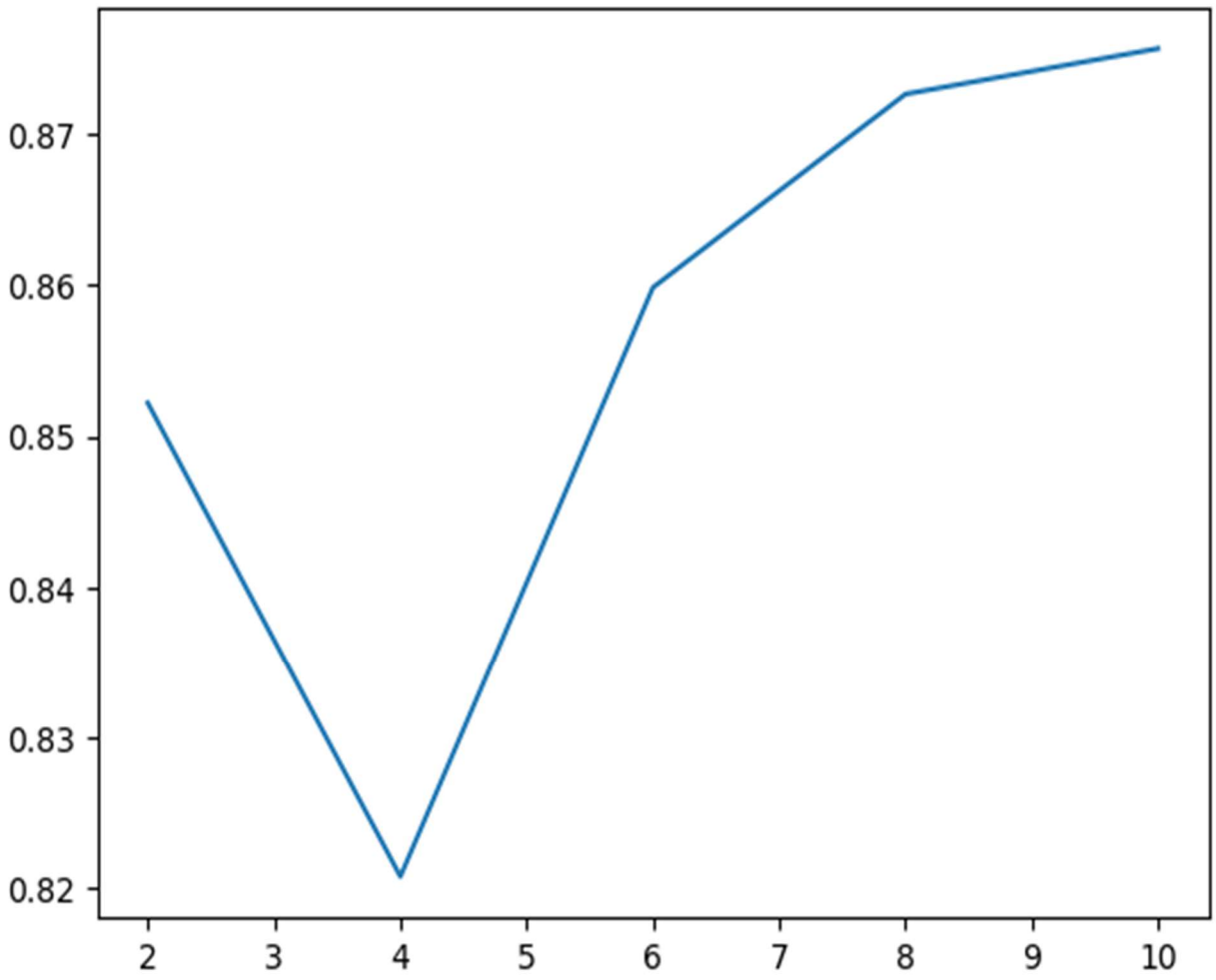
```
[0.9999996532619156,
 0.9999999944087113,
 0.9999999995081273,
 0.9999999999655694,
 0.9999999999940308]
```

```
test_scores_df=pd.DataFrame(algo_test_scores)
train_scores_df=pd.DataFrame(algo_train_scores)

test_scores_df["GBM"].plot()

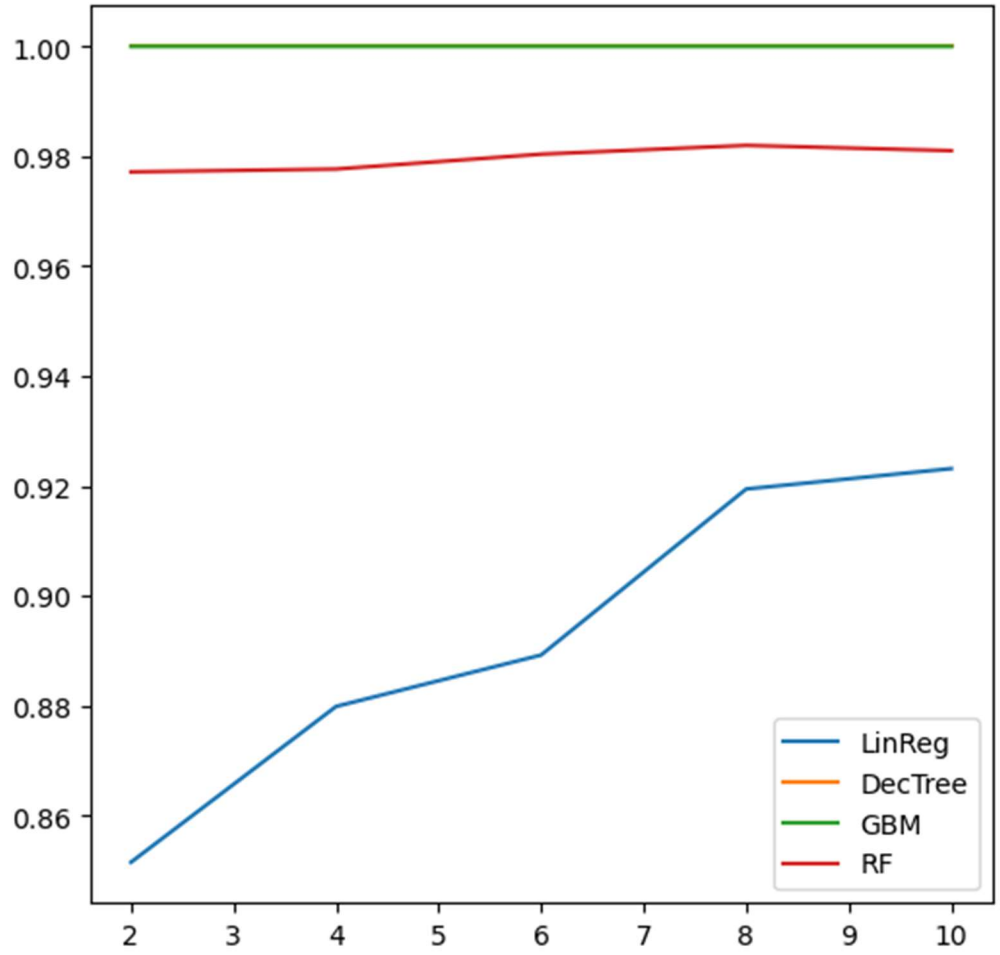
plt.show()
```

```
<Axes: >
```



```
train_scores_df.plot(figsize=(6,6))
plt.show()
```

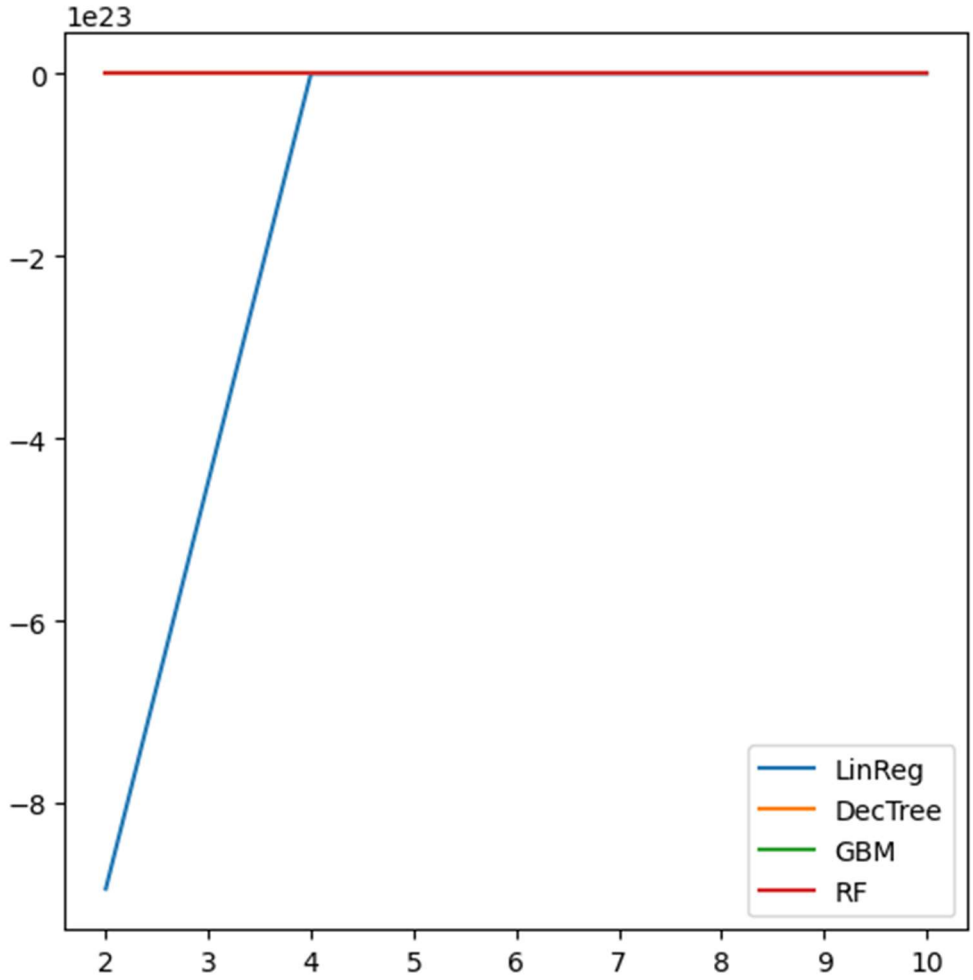
```
<Axes: >
```



```
test_scores_df.plot(figsize=(6,6))

plt.show()
```

```
<Axes: >
```




```
DecTree_1=DecisionTreeRegressor()
GBM_1=GradientBoostingRegressor(n_estimators=3000)
LinReg_1=LinearRegression()
RandFor_1=RandomForestRegressor(n_estimators=5000)

algo_dict_1 =
{"LinReg":LinReg_1,"DecTree":DecTree_1,"GBM":GBM_1,"RF":RandFor_1}
algo_train_scores_1={}
algo_test_scores_1={}
```

```
max_kfold = 10
for algo_name in algo_dict_1.keys():
    print(algo_name)
    train_score, test_score =
train_test_evaluate_with_kFold(housetraindf1,max_kfold+1,algo_dict_1[algo_nam
e])
    algo_train_scores_1[algo_name]=train_score
    algo_test_scores_1[algo_name]=test_score

print(algo_train_scores_1)
print(algo_test_scores_1)
```

```
LinReg
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
DecTree
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
GBM
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
RF
kvalue:  2
kvalue:  4
kvalue:  6
kvalue:  8
kvalue:  10
{'LinReg': {2: 0.8515525568159341, 4: 0.8798918160406497, 6:
0.8892507857704975, 8: 0.9194573118782721, 10: 0.9231709101403272},
'DecTree': {2: 1.0, 4: 1.0, 6: 1.0, 8: 1.0, 10: 1.0}, 'GBM': {2:
0.9999996532619156, 4: 0.9999999944087113, 6: 0.9999999995081273, 8:
0.9999999999655694, 10: 0.9999999999940308}, 'RF': {2: 0.9771561418658807, 4:
0.9777937155103115, 6: 0.9805079146338928, 8: 0.9819892356327434, 10:
0.9811353616699557}}
{'LinReg': {2: -8.940759370531631e+23, 4: -339.63965657330226, 6:
0.5283525768819836, 8: 0.8699774994416338, 10: 0.8608249354369647},
'DecTree': {2: 0.6993299969188821, 4: 0.7187917569358868, 6:
0.7359318076475204, 8: 0.7019080117146099, 10: 0.7508779840220209}, 'GBM':
{2: 0.8520937109835213, 4: 0.8113666411883794, 6: 0.8602579914299135, 8:
0.8740450487243151, 10: 0.8761762529046744}, 'RF': {2: 0.8524970511186785, 4:
```

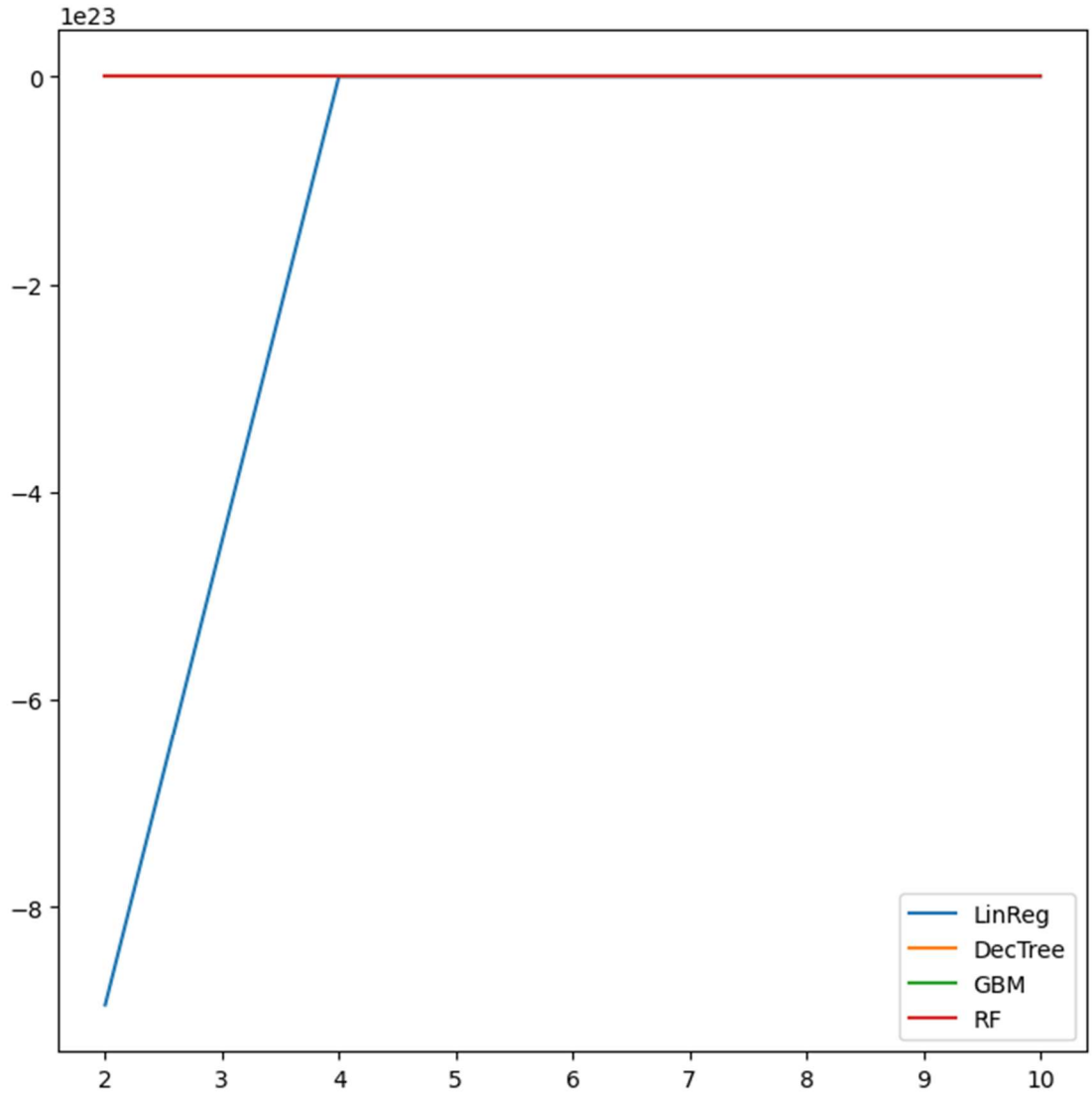
```
0.8299392139702989, 6: 0.85860237127662, 8: 0.8682083727111976, 10:
0.8545367698367154}}

test_scores_df_1=pd.DataFrame(algo_test_scores_1)
train_scores_df_1=pd.DataFrame(algo_train_scores_1)

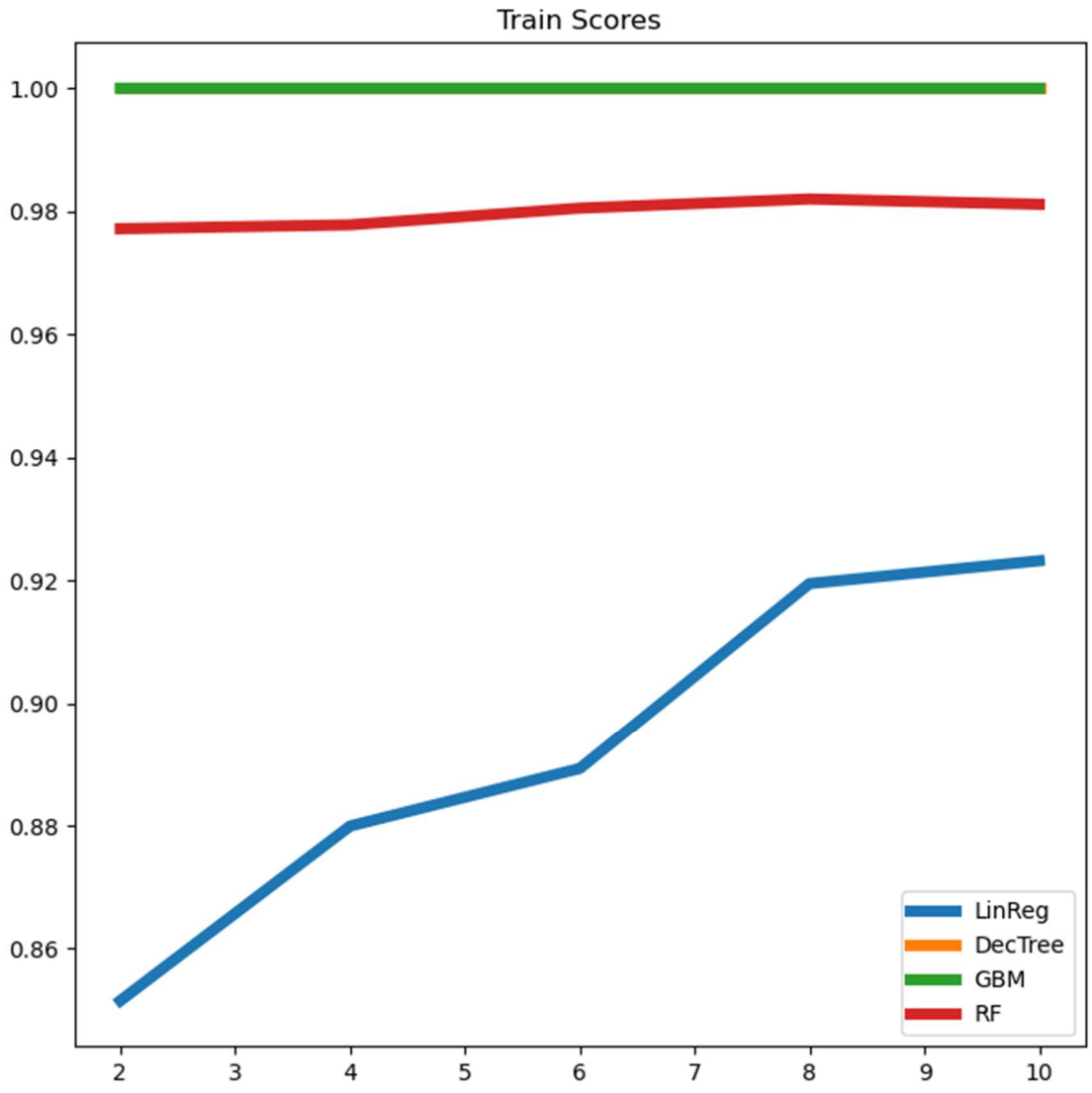
test_scores_df_1.plot(figsize=(8,8))

plt.show()
```

```
<Axes: >
```

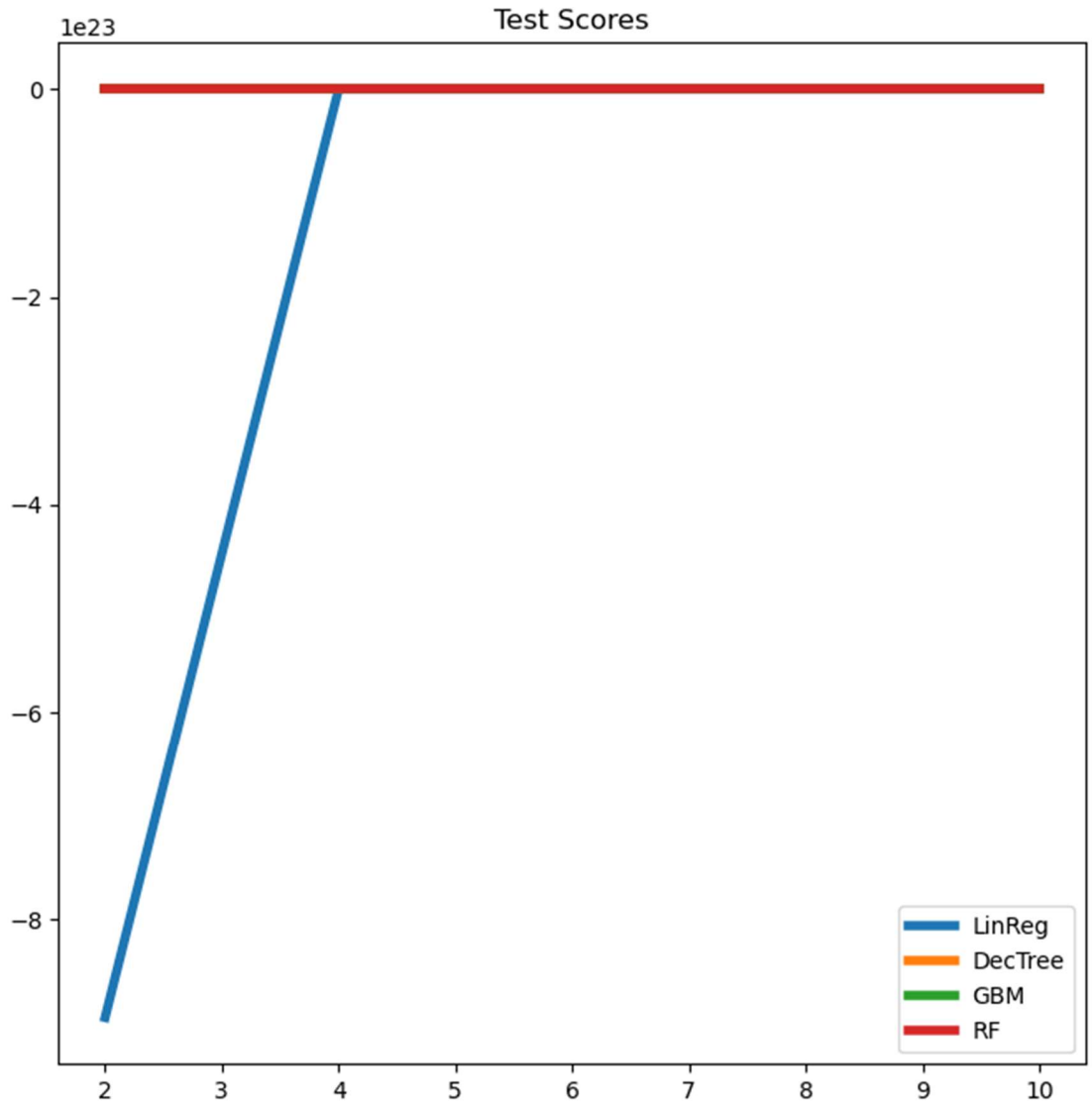


```
train_scores_df_1.plot(lw=5,figsize=(8,8),title="Train Scores")
plt.show()
<Axes: title={'center': 'Train Scores'}>
```



```
test_scores_df_1.plot(figsize=(8,8),lw=4,title="Test Scores")
plt.show()
```

```
<Axes: title={'center': 'Test Scores'}>
```



train_scores_df_1

	LinReg	DecTree	GBM	RF
2	0.851553	1.0	1.0	0.977156
4	0.879892	1.0	1.0	0.977794
6	0.889251	1.0	1.0	0.980508
8	0.919457	1.0	1.0	0.981989
10	0.923171	1.0	1.0	0.981135

test_scores_df_1

	LinReg	DecTree	GBM	RF	
2	-8.940759e+23	0.699330	0.852094	0.852497	
4	-3.396397e+02	0.718792	0.811367	0.829939	
6	5.283526e-01	0.735932	0.860258	0.858602	
8	8.699775e-01	0.701908	0.874045	0.868208	
10	8.608249e-01	0.750878	0.876176	0.854537	